

Capítulo 10 — Planificación y Razonamiento

Sección 01: Introducción a la planificación y el razonamiento

Los capítulos anteriores de este módulo establecieron los bloques fundamentales del Context Engineering: cómo construir agentes, cómo coordinarlos y cómo gestionar la memoria que necesitan para operar con continuidad. Todo ese andamiaje, sin embargo, depende de una capacidad subyacente que aún no hemos examinado en profundidad: la capacidad del modelo de lenguaje para razonar sobre una tarea antes de actuar sobre ella.

Razonar, en el contexto de un LLM, no significa lo mismo que razonar en el sentido filosófico o matemático. Significa algo más específico y, para el AI Engineer, más útil: la capacidad de descomponer un problema en pasos intermedios, evaluar alternativas, detectar errores en la propia respuesta y revisar el plan antes de continuar. Estas capacidades no son propiedades intrínsecas del modelo; son propiedades que emergen cuando el arquitecto del sistema diseña el contexto correctamente.

Este capítulo estudia cómo funciona ese proceso desde adentro: qué ocurre cuando un LLM genera una cadena de pensamiento, qué arquitecturas de planificación existen y cuándo conviene usar cada una, cómo un agente puede reflexionar sobre su propio output y corregirlo, y cómo verificar los resultados de un sistema de razonamiento antes de que esos resultados lleguen al usuario o a otro sistema.

Por qué este capítulo es diferente al capítulo 08

En el capítulo 08 se estudió el ciclo de vida del agente: percepción, planificación, acción y observación. La planificación aparecía allí como una etapa del ciclo, pero no se analizaba su estructura interna. El capítulo 10 entra en esa estructura. La pregunta no es qué hace el agente cuando planifica, sino cómo lo hace a nivel de llamadas al modelo, de diseño de prompt y de arquitectura de contexto.

La distinción es análoga a la diferencia entre describir que un motor de combustión produce movimiento (el ciclo del agente) y explicar cómo funciona el ciclo termodinámico interno (los

mecanismos de razonamiento). Ambas perspectivas son necesarias. El capítulo 08 daba la primera; el capítulo 10 da la segunda.

Qué va a encontrar el lector en este capítulo

El capítulo está organizado en cuatro bloques:

Bloque conceptual (secciones 01 a 03): qué significa razonar en un LLM, en qué se diferencia del razonamiento simbólico formal, y cuál es la taxonomía de patrones de planificación disponibles para el arquitecto de sistemas.

Bloque de técnicas (secciones 04 a 07): las técnicas concretas que implementan esos patrones — Chain of Thought, Tree of Thoughts, planificación iterativa, reflexión y verificación — con análisis de cuándo usar cada una, qué cuesta y qué riesgos presenta.

Bloque de aplicación (secciones 08 a 10): cómo estas técnicas se integran en arquitecturas empresariales reales, qué patrones funcionan y cuáles fallan en producción, y un caso de estudio completo que ilustra las decisiones de diseño en un contexto concreto.

Bloque de cierre (secciones 11 a 15): laboratorio práctico, checklist del AI Engineer, resumen, autoevaluación y transición al capítulo 11.

Nivel de abstracción

Este capítulo trabaja al nivel de patrones de diseño, no de implementación de código. El objetivo es que el lector entienda la estructura de los mecanismos de razonamiento con suficiente profundidad para tomar decisiones de arquitectura correctas: cuándo un sistema necesita razonamiento multi-paso, cuándo la reflexión agrega valor y cuándo no, qué estrategia de verificación corresponde a cada tipo de output.

Los diagramas de flujo de llamadas al modelo reemplazan aquí a los fragmentos de código. Son más generalizables que cualquier SDK particular y capturan la lógica arquitectónica que permanece estable aunque las bibliotecas cambien.

La siguiente sección establece el fundamento conceptual imprescindible: qué significa, con precisión, que un LLM razone.

Capítulo 10 — Planificación y Razonamiento

Sección 02: Qué significa razonar en un LLM

Antes de diseñar un sistema de razonamiento, es necesario entender con precisión qué ocurre cuando un LLM produce lo que parece ser una cadena de razonamiento. Esta distinción conceptual no es un ejercicio filosófico: tiene consecuencias directas sobre las decisiones de arquitectura y sobre lo que el ingeniero puede esperar del sistema.

Razonamiento simbólico versus predicción de tokens

El razonamiento simbólico formal — el de la lógica deductiva, la demostración matemática o el planificador clásico de IA — opera sobre símbolos con semántica fija. Dado un conjunto de premisas y reglas de inferencia, el sistema produce conclusiones que son necesariamente correctas si las premisas son correctas. El proceso es determinista, verificable y explicable paso a paso.

Un LLM no opera de esa manera. Un modelo de lenguaje es, en su núcleo, una función de probabilidad condicional: dado el texto que precede al siguiente token, el modelo asigna una distribución de probabilidad sobre todos los tokens posibles y selecciona uno (o muestrea uno, según la temperatura configurada). Este proceso se repite token a token hasta que el modelo genera la secuencia completa de la respuesta.

No hay inferencia formal. No hay motor de reglas. No hay manipulación de símbolos con semántica garantizada. Lo que hay es una función estadística extremadamente compleja, entrenada sobre cantidades masivas de texto humano, que aprendió a producir secuencias de tokens que son consistentes con el contexto dado.

Cuando ese contexto incluye problemas con soluciones correctas — matemáticas, programación, razonamiento lógico — el modelo produce secuencias de tokens que, estadísticamente, se asemejan a las soluciones correctas que aparecían en sus datos de entrenamiento y en el contexto inmediato. El output parece razonado porque el razonamiento humano tiene patrones lingüísticos reconocibles, y el modelo los reproduce.

Por qué esto importa para el diseño

Esta distinción tiene tres consecuencias prácticas inmediatas:

Primera consecuencia: el modelo puede equivocarse con plena confianza. Cuando un LLM afirma algo incorrecto, no está mintiendo ni fallando deliberadamente. Está produciendo la secuencia de tokens más probable dado su contexto, y esa secuencia puede ser factualmente incorrecta. No existe, en el modelo, un mecanismo que detecte la contradicción entre lo que afirma y la verdad del mundo. El arquitecto del sistema es quien debe añadir ese mecanismo externamente.

Segunda consecuencia: el contexto determina la calidad del razonamiento. Dado que el modelo predice tokens condicionado al contexto, la calidad del razonamiento que produce depende directamente de cómo se estructura ese contexto. Un prompt que incluye pasos de razonamiento intermedios induce al modelo a continuar con pasos similares. Un prompt que pregunta directamente por la respuesta final induce al modelo a saltar a ella, potencialmente omitiendo verificaciones necesarias. El diseño del contexto es el principal lever de control que tiene el AI Engineer sobre la calidad del razonamiento.

Tercera consecuencia: el razonamiento no es introspección. Cuando un modelo explica cómo llegó a una conclusión, esa explicación es una secuencia de tokens que el modelo genera después (o simultáneamente con) la conclusión. No es una descripción fiel del proceso computacional que produjo la respuesta. El modelo no tiene acceso privilegiado a sus propios pesos. La explicación es, también, predicción estadística. Esto tiene implicaciones importantes para los sistemas de verificación: no se puede confiar en que el modelo detecte sus propios errores simplemente pidiéndole que los explique.

El razonamiento emergente y cómo aprovecharlo

Si el razonamiento de un LLM es predicción estadística, ¿qué lo hace útil en la práctica?

Lo que ocurre es que ciertos patrones de razonamiento son lo suficientemente frecuentes y consistentes en el texto humano que el modelo los reproduce con alta fidelidad. La lógica deductiva básica, la descomposición de problemas en subproblemas, la identificación de contradicciones, el reconocimiento de patrones en código — todo esto aparece repetidamente en el corpus de entrenamiento, y el modelo aprende a reproducir esos patrones de forma confiable.

Esto significa que el razonamiento de un LLM es robusto en los dominios donde el modelo tiene buena cobertura de entrenamiento y donde el razonamiento tiene patrones lingüísticos

reconocibles. Y es frágil en problemas novedosos, en razonamiento numérico preciso sin herramientas de apoyo, o en cualquier dominio donde el modelo no tuvo exposición suficiente durante el entrenamiento.

La clave para el AI Engineer es no tratar al modelo como una caja de razonamiento general, sino como un sistema que, bajo las condiciones correctas de contexto, produce outputs de calidad razonable en dominios donde ha sido bien entrenado. El diseño del sistema debe complementar las capacidades del modelo con mecanismos externos — herramientas, verificación, reflexión — donde el modelo solo no es suficientemente confiable.

Diagrama conceptual

```
RAZONAMIENTO SIMBÓLICO FORMAL

Premisas → Reglas de inferencia → Conclusión (garantizada)

Determinista | Verificable | Explicable


RAZONAMIENTO EN LLM

Contexto → Predicción token-a-token → Secuencia de output

Probabilístico | Estadístico | Emergente del entrenamiento


LO QUE EL ARQUITECTO CONTROLA:

- El contenido y estructura del contexto

- Los pasos intermedios que el modelo debe producir

- La verificación externa del output

- Las herramientas disponibles para complementar al modelo
```

Nota del arquitecto

El error más costoso en la práctica no es sobreestimar las capacidades del modelo — los ingenieros con experiencia aprenden rápido a desconfiar de respuestas no verificadas. El error más costoso es infrautilizar el contexto: tratar al modelo como un oráculo que solo necesita la pregunta, cuando en realidad es un sistema que requiere que el contexto estructure el proceso de razonamiento. Un modelo mediocre con un contexto bien diseñado supera consistentemente a un modelo superior con un contexto pobre.

La siguiente sección presenta la taxonomía de patrones de planificación que el arquitecto puede usar para estructurar ese contexto de forma efectiva.

Capítulo 10 — Planificación y Razonamiento

Sección 03: Patrones modernos de planificación

Dado que el razonamiento de un LLM depende del contexto, la pregunta del arquitecto es: ¿qué estructura de contexto induce el mejor razonamiento para cada tipo de tarea? La respuesta es una taxonomía de patrones de planificación. Cada patrón define cómo se estructura el flujo de llamadas al modelo, qué se incluye en el contexto de cada llamada y cómo se integran los outputs intermedios.

Esta taxonomía tiene cuatro patrones fundamentales. Son suficientemente estables como principios de diseño, aunque sus implementaciones específicas evolucionen con las herramientas y los modelos.

Patrón 1: Llamada simple

Descripción: Una sola llamada al modelo produce la respuesta completa. El contexto contiene la tarea completa, y el modelo genera el output en un único paso.

Estructura:

```
[contexto: sistema + tarea] → [llamada al modelo] → [output final]
```

Cuándo usarlo: Tareas simples, bien delimitadas, donde el espacio de respuestas correctas es acotado. Clasificación de texto, extracción de entidades, transformación de formato, respuesta a preguntas con respuesta directa en el corpus de entrenamiento.

Costo computacional: Mínimo. Una llamada al modelo, latencia de una inferencia.

Limitaciones: Para problemas complejos, el modelo tiene a saltar pasos intermedios de razonamiento, lo que aumenta la tasa de error. No hay oportunidad de corrección intermedia. La calidad del output depende enteramente de que el modelo produzca la respuesta correcta en un solo intento.

Ejemplo: Un sistema de clasificación de tickets de soporte técnico que categoriza cada ticket en una de ocho categorías predefinidas. La tarea es suficientemente simple y el modelo suficientemente bien entrenado en el dominio como para que una sola llamada sea el diseño correcto.

Patrón 2: Planificación secuencial (cadena de llamadas)

Descripción: La tarea se descompone en pasos, y cada paso es una llamada separada al modelo. El output de cada llamada se incorpora al contexto de la siguiente.

Estructura:

```
[tarea] → [llamada 1: descomposición] → [plan de pasos]
      → [llamada 2: paso 1] → [resultado parcial 1]
      → [llamada 3: paso 2, contexto incluye resultado 1] → [resultado parcial 2]
      → [llamada N: síntesis] → [output final]
```

Cuándo usarlo: Tareas que tienen estructura natural de pipeline: análisis seguido de síntesis, búsqueda seguida de respuesta, traducción de formato seguida de validación. También útil cuando la tarea supera la capacidad de razonamiento del modelo en una sola llamada.

Costo computacional: N veces la latencia de una llamada, donde N es el número de pasos. El contexto crece en cada llamada al incluir los resultados anteriores.

Limitaciones: Los errores en pasos tempranos se propagan a los pasos siguientes sin posibilidad de corrección automática. No hay ramificación: si el plan inicial es incorrecto, el sistema lo sigue hasta el final.

Ejemplo: Un sistema de análisis de contratos que primero extrae las cláusulas relevantes (llamada 1), luego identifica las obligaciones de cada parte en cada cláusula (llamada 2), luego evalúa el riesgo de cada obligación (llamada 3) y finalmente produce un resumen ejecutivo (llamada 4).

Patrón 3: Planificación iterativa (con reflexión)

Descripción: El modelo genera una respuesta, luego la evalúa (en la misma llamada o en una llamada separada), identifica problemas y produce una versión mejorada. El ciclo se repite hasta que el output satisface un criterio de calidad o se alcanza un límite de iteraciones.

Estructura:

```
[tarea] → [llamada 1: borrador] → [respuesta inicial]
      → [llamada 2: evaluación] → [crítica + lista de problemas]
      → [llamada 3: revisión] → [respuesta mejorada]
      → [llamada 4: evaluación] → [¿satisfactorio? sí → fin / no → continuar]
```

Cuándo usarlo: Tareas donde la calidad del output es crítica y los errores son costosos. Generación de código que debe funcionar. Análisis que debe ser exhaustivo. Respuestas que serán presentadas a clientes o tomadas de decisiones de negocio. Cualquier tarea donde el modelo comete errores identificables con un segundo intento.

Costo computacional: Variable. Típicamente 2x a 4x el costo de una llamada simple. En el peor caso, puede generar muchas iteraciones sin convergencia si el criterio de evaluación no está bien definido. Es necesario establecer un límite de iteraciones.

Limitaciones: El mismo modelo que generó el error es el que lo evalúa. Hay clases de errores que el modelo no puede detectar por sí mismo — en particular, errores factuales en dominios donde el modelo tiene conocimiento incorrecto. La reflexión mejora la coherencia y la completitud, pero no garantiza la corrección factual.

Ejemplo: Un sistema de generación de código que produce una función, luego evalúa si maneja todos los casos borde descritos en el enunciado, luego revisa el código para agregar los casos que faltaban, y finalmente verifica que la versión revisada es sintácticamente correcta y completa.

Patrón 4: Planificación ramificada (Tree of Thoughts)

Descripción: El modelo genera múltiples ramas de razonamiento en paralelo, evalúa cada rama y selecciona la más prometedora para continuar. El proceso se repite hasta alcanzar una solución o un límite de profundidad.

Estructura:

```
[tarea]
→ [llamada 1a: enfoque A] → [razonamiento A]
→ [llamada 1b: enfoque B] → [razonamiento B]
→ [llamada 1c: enfoque C] → [razonamiento C]
→ [llamada 2: evaluación de A, B, C] → [selección: B es más prometedor]
→ [llamada 3a: continuación de B, rama B1]
→ [llamada 3b: continuación de B, rama B2]
→ [llamada 4: evaluación B1 vs B2] → [selección: B1]
→ [llamada 5: solución final desde B1] → [output final]
```

Cuándo usarlo: Problemas de optimización donde el espacio de soluciones es grande y la solución óptima no es evidente. Tareas creativas donde hay múltiples enfoques válidos y se busca el mejor. Problemas donde el primer enfoque intuitivo del modelo frecuentemente es subóptimo.

Costo computacional: Alto. El número de llamadas crece exponencialmente con la profundidad del árbol y el factor de ramificación. En práctica, se usan árboles poco profundos (2-3 niveles) con ramificación limitada (2-3 ramas). Para una configuración típica de 3 niveles con 3 ramas, se necesitan del orden de 40 llamadas al modelo.

Limitaciones: Latencia alta. Costo alto. Complejidad de implementación alta. Solo justificado cuando la calidad del output tiene un valor suficientemente alto como para justificar el costo. No escala bien para sistemas de alto throughput.

Ejemplo: Un sistema de generación de estrategias de negocio que genera tres enfoques estratégicos diferentes para un problema de mercado, evalúa cuál tiene mayor viabilidad dado el contexto de la empresa, desarrolla el enfoque seleccionado con detalle en dos variantes de implementación, evalúa cuál variante tiene menor riesgo y mayor impacto, y finalmente produce el plan detallado de la variante ganadora.

Tabla de selección de patrones

Criterio	Simple	Secuencial	Iterativo	Ramificado
Complejidad de la tarea	Baja	Media	Media-alta	Alta
Calidad requerida	Suficiente	Buena	Alta	Óptima
Latencia admisible	Mínima	Moderada	Alta	Muy alta
Costo admisible	Mínimo	Moderado	Alto	Muy alto
Errores propagables	No aplica	Riesgo	Controlado	Controlado

Nota del arquitecto

Los patrones no son mutuamente excluyentes. Un sistema de producción típico usa una combinación: planificación secuencial como estructura base, con reflexión iterativa en los pasos donde la calidad es crítica, y llamada simple para los pasos de bajo riesgo. La decisión de qué patrón usar en cada paso es una decisión de ingeniería que balancea calidad, latencia y costo — los tres ejes que definen el espacio de diseño de sistemas de razonamiento.

La siguiente sección examina en detalle las técnicas que implementan los patrones 2 y 4: Chain of Thought y Tree of Thoughts.

Capítulo 10 — Planificación y Razonamiento

Sección 04: Chain of Thought, Tree of Thoughts y variantes

Las técnicas de esta sección son las implementaciones más influyentes de los patrones de planificación secuencial y ramificado. Aunque el Módulo 2 introdujo Chain of Thought como técnica de prompting, aquí se examina desde una perspectiva diferente y más relevante para el AI Engineer: no como técnica de prompting, sino como mecanismo de diseño del contexto en sistemas de agentes.

Chain of Thought: del prompting a la arquitectura

La perspectiva del Módulo 2 trató Chain of Thought (CoT) como una instrucción al modelo: incluir en el prompt la indicación de pensar paso a paso, o incluir ejemplos few-shot donde la respuesta mostraba el razonamiento intermedio. Eso es suficiente para un prompt individual.

La perspectiva arquitectónica va más allá. En un sistema de agentes, Chain of Thought no es una instrucción en un prompt: es la decisión de diseño de separar el razonamiento de la respuesta en llamadas distintas, de materializar los pasos intermedios como outputs que se incorporan al contexto de las llamadas siguientes.

Esta distinción importa porque tiene consecuencias en la observabilidad del sistema, en la capacidad de intervención y en la distribución de errores.

Anatomía de un CoT arquitectónico:

PASO 1 — LLAMADA DE RAZONAMIENTO

Prompt: "Eres un analista financiero. Aquí está el balance de una empresa:

[datos financieros]. Antes de emitir cualquier conclusión, identifica:

- (a) los tres indicadores más relevantes para el sector,
- (b) los valores de cada indicador en el periodo actual y anterior,
- (c) la tendencia de cada indicador.

Responde solo con este análisis, sin conclusiones."

Output del modelo:

- (a) Indicadores relevantes: ratio de liquidez, margen EBITDA, deuda/patrimonio.
- (b) Ratio liquidez: 1.8 actual vs 2.3 anterior. Margen EBITDA: 12% vs 15%.
Deuda/patrimonio: 0.65 vs 0.48.
- (c) Tendencias: liquidez deteriorándose, margen comprimiéndose, apalancamiento aumentando.

PASO 2 — LLAMADA DE CONCLUSIÓN

Prompt: [sistema anterior] + [output del paso 1] + "Con base en el análisis anterior, emite una evaluación de riesgo crediticio (bajo/medio/alto) y justifica brevemente."

Output del modelo:

Evaluación: Riesgo medio-alto. La empresa muestra deterioro simultáneo en tres indicadores clave durante el periodo. El aumento del apalancamiento combinado con la compresión del margen reduce la capacidad de servicio de deuda.

Qué ganó la separación en dos llamadas:

1. El paso 1 es auditable de forma independiente. El equipo puede revisar si el análisis de indicadores es correcto sin revisar la conclusión.
2. Si el paso 1 comete un error, ese error es visible y puede corregirse antes de que llegue al paso 2.
3. El sistema puede configurarse para requerir aprobación humana entre los pasos, lo que es relevante en entornos con regulación o supervisión.
4. Los errores del paso 2 son más fáciles de diagnosticar porque el contexto que recibe es explícito y verificable.

Cuándo Chain of Thought mejora la calidad y cuándo no:

CoT mejora la calidad cuando la tarea tiene estructura de razonamiento multi-paso y el modelo tiende a saltar pasos intermedios en ausencia de instrucción explícita. Mejora la precisión en matemáticas aplicadas, análisis lógico, diagnóstico de problemas de código y análisis de múltiples factores.

CoT no mejora la calidad — y puede perjudicarla — en tareas de recuperación directa de información, en clasificación simple donde el modelo ya tiene alta precisión, o en tareas creativas donde la fluidez del razonamiento lineal inhibe la generación de ideas no convencionales. Un CoT mal diseñado puede también guiar al modelo por un camino de razonamiento incorrecto y hacer que llegue a una conclusión incorrecta con mayor confianza que si hubiera respondido directamente.

Tree of Thoughts: razonamiento como búsqueda

Tree of Thoughts (ToT) es una arquitectura publicada en 2023 que formaliza el patrón de planificación ramificado. Su intuición central es que el razonamiento puede modelarse como un proceso de búsqueda sobre un árbol de estados, donde cada nodo es un pensamiento intermedio y los arcos son los pasos de razonamiento que conectan pensamientos.

Los tres componentes del ToT:

- 1. Generación de pensamientos:** En cada nodo del árbol, el modelo genera múltiples pensamientos posibles. Un "pensamiento" es un fragmento coherente de razonamiento — una hipótesis, un enfoque, un paso de solución — que tiene sentido analizar de forma independiente.
- 2. Evaluación de pensamientos:** Cada pensamiento generado se evalúa. La evaluación puede ser mediante una llamada separada al mismo modelo ("¿es este enfoque prometedor?"), mediante una función heurística diseñada para el dominio, o mediante ejecución directa (en el caso de código, verificar si compila y pasa tests básicos).
- 3. Selección y continuación:** Los pensamientos con mejor evaluación se seleccionan para continuar. Los pensamientos descartados se podan. El proceso se repite hasta alcanzar una solución completa o un límite de profundidad.

Ejemplo concreto — Diseño de una estrategia de migración de datos:

```
NODO RAÍZ: "Diseñar estrategia para migrar 50 TB de datos de clientes de
sistema on-premise a cloud sin downtime."
```

```
NIVEL 1 — GENERACIÓN (3 pensamientos):
```

- ```
A: "Migración por replicación continua: mantener ambos sistemas en sync
 durante la transición."
B: "Migración por lotes nocturnos: procesar los datos fuera de horario pico
 durante 4 semanas."
C: "Migración Blue-Green: crear entorno paralelo, migrar datos, hacer
 cutover atómico."
```

```
NIVEL 1 — EVALUACIÓN:
```

- ```
A: Viable pero compleja de mantener; riesgo de divergencia de datos.
B: Simple pero implica 28 días de datos en migración; riesgo acumulado.
C: Más limpia para el cutover; requiere doble infraestructura durante transición.
→ Seleccionado: C (mejor balance riesgo/complejidad para el requisito de no downtime)
```

```
NIVEL 2 — GENERACIÓN (desde C, 2 variantes):
```

- ```
C1: "Blue-Green con sincronización unidireccional (on-premise → cloud) durante
 el periodo de transición."
C2: "Blue-Green con sincronización bidireccional para soporte de rollback completo."
```

```
NIVEL 2 — EVALUACIÓN:
```

- ```
C1: Más simple; rollback requiere re-sincronizar en dirección inversa.
C2: Rollback inmediato; pero doble la complejidad de sincronización.
→ Seleccionado: C2 si el requisito de rollback es crítico; C1 en caso contrario.
```

```
OUTPUT FINAL: Plan detallado para la variante seleccionada con estimación de
recursos, duración y criterios de cutover.
```

El costo real del Tree of Thoughts:

Para un árbol de profundidad 2 con factor de ramificación 3, el número de llamadas es:

- Nivel 1: 3 llamadas de generación + 1 llamada de evaluación = 4 llamadas

- Nivel 2: 2 llamadas de generación + 1 llamada de evaluación = 3 llamadas (continuando solo la rama seleccionada)
- Output final: 1 llamada
- Total: 8 llamadas

Si el factor de ramificación es 3 en todos los niveles y se evalúan todas las ramas en todos los niveles antes de podar:

- Nivel 1: 3 generaciones + 1 evaluación = 4
- Nivel 2: 9 generaciones + 1 evaluación = 10
- Total: 14+ llamadas

La latencia total es la suma de las latencias de todas las llamadas. Para sistemas de alto volumen o con requisitos de respuesta en tiempo real, ToT completo raramente es práctico. En la práctica, se usan versiones podadas agresivamente — 2 niveles máximo, 2-3 ramas, poda temprana.

Variantes notables

Self-Consistency: Genera múltiples cadenas de razonamiento independientes para el mismo problema (varias llamadas paralelas con temperatura > 0) y selecciona la respuesta más frecuente entre ellas. Mejora la precisión en problemas con respuesta discreta (matemáticas, clasificación) sin la complejidad de la arquitectura ToT completa.

ReAct (Reasoning + Acting): Intercala pasos de razonamiento con acciones sobre herramientas. El modelo razona sobre qué herramienta usar, usa la herramienta, observa el resultado, razona sobre qué hacer a continuación. Es el patrón más común en agentes con herramientas porque combina razonamiento y ejecución en un ciclo natural.

Plan-and-Execute: Separa explícitamente la fase de planificación de la fase de ejecución. Una llamada inicial produce el plan completo; llamadas subsiguientes ejecutan cada paso del plan. La ventaja es que el plan puede ser revisado por un humano o por otro agente antes de ejecutarse.

Nota del arquitecto

La elección entre CoT, ToT o sus variantes no es principalmente técnica: es económica. El AI Engineer debe estimar el valor del output (¿cuánto cuesta un error en producción?) y

compararlo con el costo de la arquitectura de razonamiento (llamadas adicionales, latencia, complejidad de mantenimiento). Para la mayoría de los casos de uso empresariales, CoT arquitectónico con un paso de evaluación es el balance correcto. ToT completo se reserva para decisiones de alto valor donde el costo del error supera ampliamente el costo de las llamadas adicionales.

La siguiente sección examina cómo la planificación iterativa — el patrón 3 — implementa estos principios en un ciclo de ejecución continua.

Capítulo 10 — Planificación y Razonamiento

Sección 05: Planificación iterativa y ejecución

Los patrones de planificación de la sección anterior son estructuras estáticas: el arquitecto define la secuencia de llamadas de antemano. La planificación iterativa es diferente: el agente decide en tiempo de ejecución qué paso dar a continuación, basándose en el resultado del paso anterior. Esta capacidad — planificar dinámicamente a medida que avanza — es lo que distingue a un agente con razonamiento real de un pipeline fijo.

El ciclo de planificación dinámica

Un agente con planificación iterativa opera en un ciclo que se repite hasta que la tarea está completa o se alcanza un límite:

ESTADO INICIAL

Tarea: [descripción de la tarea]

Herramientas disponibles: [lista de herramientas]

Contexto: [información disponible]

ITERACIÓN N

1. RAZONAMIENTO: "Dado el estado actual, ¿cuál es el próximo paso necesario?"

2. SELECCIÓN DE ACCIÓN: elegir entre:

- Usar una herramienta
- Consultar al usuario
- Generar un output intermedio
- Declarar la tarea completa

3. EJECUCIÓN: realizar la acción seleccionada

4. OBSERVACIÓN: incorporar el resultado al contexto

5. EVALUACIÓN: ¿la tarea está completa? → si sí, terminar / si no, continuar

ESTADO FINAL

Output: [resultado de la tarea]

Traza: [historia de pasos, acciones y observaciones]

Este ciclo es la implementación concreta del patrón ReAct (Reasoning + Acting) mencionado en la sección anterior. La característica clave es que cada iteración produce información nueva que modifica el plan para la siguiente iteración.

Por qué la planificación iterativa falla sin control

La planificación iterativa es poderosa pero introduce un riesgo que el arquitecto debe gestionar explícitamente: el ciclo puede no terminar.

El problema del loop infinito de planificación ocurre cuando el agente:

- Genera pasos que no avanzan hacia la solución (da vueltas alrededor del mismo problema)
- Alcanza un estado donde no puede avanzar porque una herramienta falla y no tiene alternativa
- Malinterpreta el criterio de completitud y sigue generando pasos innecesarios después de que la tarea está resuelta

- Genera un plan que requiere herramientas no disponibles y queda atrapado intentando diferentes variantes

Los mecanismos de control que el sistema debe implementar:

1. Límite de iteraciones: El número máximo de ciclos que el agente puede ejecutar antes de reportar un error o escalar al humano. Un valor típico es entre 10 y 30 iteraciones para tareas de complejidad media.

2. Detección de estancamiento: Si el agente ejecuta la misma acción (o acciones muy similares) más de N veces sin avanzar, el sistema debe interrumpir el ciclo y escalar.

3. Criterio de completitud explícito: El prompt del agente debe incluir una descripción precisa de cuándo la tarea está completa. Sin esta descripción, el modelo puede continuar generando pasos adicionales que degradan el output en lugar de mejorarlo.

4. Timeout: Un límite de tiempo absoluto, independiente del número de iteraciones, que garantiza que el sistema no bloquea indefinidamente.

Plan-and-Execute: separar pensar de actuar

Una variante más controlable de la planificación iterativa es el patrón Plan-and-Execute. En lugar de decidir el próximo paso en cada iteración, el agente genera primero un plan completo y luego lo ejecuta paso a paso.

Estructura:

FASE 1 — PLANIFICACIÓN (una llamada al modelo)

Prompt: "Dada la tarea [descripción], genera un plan paso a paso para completarla. Para cada paso, especifica: (a) la acción a realizar, (b) la herramienta necesaria, (c) el output esperado, (d) cómo saber si el paso fue exitoso."

Output del modelo:

Paso 1: Buscar los últimos 3 informes anuales de la empresa.

Herramienta: búsqueda web / base de datos documental.

Output esperado: 3 documentos PDF o URLs.

Criterio de éxito: documentos con fechas correspondientes a los últimos 3 años.

Paso 2: Extraer los indicadores financieros clave de cada informe.

Herramienta: extractor de documentos.

Output esperado: tabla con indicadores por año.

Criterio de éxito: tabla completa para los 3 periodos.

[...]

FASE 2 — EJECUCIÓN (una llamada por paso)

Para cada paso del plan:

- Ejecutar la acción con la herramienta indicada
- Verificar el criterio de éxito
- Si el paso falla, intentar alternativa o escalar
- Registrar el output en la memoria de trabajo

Ventajas del Plan-and-Execute:

- El plan completo es inspeccionable antes de ejecutarse. Un humano puede revisarlo y modificarlo.
- Los errores de planificación (elegir la herramienta equivocada, olvidar un paso) se detectan antes de incurrir en el costo de ejecución.
- El progreso de ejecución es medible: "completados 3 de 7 pasos".
- Si la ejecución falla en un paso, el sistema sabe exactamente dónde está y puede replanificar desde ese punto.

Limitaciones del Plan-and-Execute:

- El plan se genera con información limitada (solo la información disponible al inicio). Si la ejecución de un paso revela información que cambiaría el plan, el sistema puede necesitar una fase de replanificación.
- El plan puede ser demasiado rígido para tareas donde los pasos dependen fuertemente de resultados intermedios que no se pueden predecir.

Replanificación dinámica

Para tareas donde el entorno es suficientemente incierto como para que el plan inicial frecuentemente requiera ajustes, se puede combinar Plan-and-Execute con un mecanismo de replanificación:

```
[Plan inicial]

→ [Ejecutar paso 1] → [Resultado]

→ [Evaluar: ¿el resultado permite continuar con el plan?]

    → Sí: continuar con el paso 2

    → No: replanificar desde el paso 2 con la información nueva
```

La replanificación es una llamada adicional al modelo que recibe: el plan original, el resultado del paso fallido (o el resultado inesperado), y la tarea original, y produce un plan revisado para los pasos restantes.

Ejemplo empresarial: análisis de proveedores

Un agente de análisis de proveedores recibe la tarea: "Evaluar si el proveedor X es un candidato adecuado para el contrato de suministro de componentes electrónicos para el próximo año."

Plan generado en la fase de planificación:

1. Buscar información pública sobre el proveedor (web, bases de datos de empresas).
2. Verificar su historial de cumplimiento en contratos similares (base de datos interna de contratos).
3. Consultar el perfil de riesgo financiero del proveedor (sistema de riesgo de crédito).
4. Verificar si el proveedor tiene certificaciones requeridas (ISO, CE, etc.).
5. Consolidar la evaluación: puntaje en cada dimensión, recomendación final.

Ejecución:

En el paso 3, el sistema de riesgo de crédito reporta que el proveedor tiene calificación "insuficiente" porque es una empresa nueva sin historial financiero previo. El plan original no contemplaba esta situación.

Replanificación: El agente replanifica a partir del paso 3: en lugar de descartar al proveedor, añade pasos adicionales para obtener referencias de clientes actuales del proveedor y verificar el respaldo financiero de su empresa matriz. La evaluación final incorpora esta información adicional.

Nota del arquitecto

La tensión entre planificación rígida y planificación dinámica es real en producción. Los sistemas con planificación completamente dinámica son difíciles de depurar porque cada ejecución puede seguir un camino diferente. Los sistemas con planificación rígida fallan cuando el entorno no coincide con los supuestos del plan. El balance práctico es Plan-and-Execute con replanificación limitada: un plan inicial revisable, con capacidad de replanificar en pasos específicos predefinidos donde el entorno es más incierto.

La siguiente sección examina el mecanismo que hace posible que el agente detecte sus propios errores y los corrija: la reflexión y autoevaluación.

Capítulo 10 — Planificación y Razonamiento

Sección 06: Reflexión y autoevaluación del agente

La reflexión es el mecanismo por el cual un agente evalúa su propio output, identifica deficiencias y genera una versión mejorada. Es uno de los patrones más importantes en sistemas de IA de producción y uno de los menos comprendidos: no porque sea difícil de implementar, sino porque requiere entender con precisión qué puede y qué no puede detectar un modelo cuando evalúa su propio trabajo.

El patrón Reflexion

El patrón Reflexion (Shinn et al., 2023) formaliza el ciclo de autoevaluación en tres fases:

Fase 1 — Generación: El agente produce una respuesta a la tarea.

Fase 2 — Evaluación: El agente (o un segundo agente evaluador) revisa la respuesta contra criterios explícitos y produce una crítica. La crítica identifica específicamente qué está mal, qué falta y qué podría mejorarse.

Fase 3 — Revisión: El agente produce una nueva respuesta incorporando la crítica. La nueva respuesta tiene acceso al contexto original, la respuesta inicial y la crítica.

El ciclo puede repetirse: la respuesta revisada puede ser evaluada nuevamente, producir una nueva crítica y generar una tercera versión. En práctica, 2-3 ciclos son el límite útil; más iteraciones raramente mejoran el output y con frecuencia lo degradan al hacer que el modelo sea excesivamente cauteloso o redundante.

Ciclo de reflexión completo: ejemplo paso a paso

Tarea: Generar una política de seguridad para el acceso a la API de un sistema de pagos.

ITERACIÓN 1 — GENERACIÓN

Prompt al agente:

Eres un especialista en seguridad de sistemas financieros. Genera una política de control de acceso para la API REST de un sistema de pagos. La política debe cubrir autenticación, autorización, rate limiting y auditoría.

Respuesta inicial:

```
POLÍTICA DE ACCESO — API DE PAGOS

1. Autenticación
  - Todos los clientes deben usar OAuth 2.0 con tokens JWT.
  - Los tokens expiran a las 24 horas.
  - Se requiere HTTPS para todas las conexiones.

2. Autorización
  - Modelo de permisos basado en roles (RBAC).
  - Roles disponibles: lectura, escritura, administración.

3. Rate Limiting
  - Máximo 1000 requests por hora por cliente.

4. Auditoría
  - Se registran todos los accesos a la API.
```

ITERACIÓN 1 — EVALUACIÓN

Prompt al evaluador:

Revisa esta política de control de acceso para una API de pagos. Identifica: (a) qué requisitos de seguridad están ausentes o incompletos, (b) qué valores o parámetros son inadecuados para un sistema financiero, (c) qué aspectos regulatorios o de compliance deben considerarse.

Crítica:

DEFICIENCIAS IDENTIFICADAS:**(a) Elementos ausentes o incompletos:**

- No especifica el algoritmo de firma del JWT ni el tamaño mínimo de clave.
- No menciona refresh tokens ni el proceso de rotación de tokens.
- El RBAC no especifica qué operaciones puede realizar cada rol.
- No hay política de bloqueo de cuentas ante intentos fallidos de autenticación.
- No se menciona validación de certificados en el cliente (mutual TLS).
- Falta política de revocación de tokens comprometidos.

(b) Valores inadecuados para sistema financiero:

- El token de 24 horas es excesivo para tokens de acceso en sistemas de pagos; PCI DSS recomienda tokens de acceso de corta vida (15-60 minutos) con refresh token separado.
- 1000 requests/hora puede ser demasiado permisivo o demasiado restrictivo según el volumen de operaciones; debe diferenciarse por operación (lectura vs. escritura vs. pago).

(c) Aspectos regulatorios ausentes:

- PCI DSS requiere retención de logs de auditoría por mínimo 12 meses.
- Debe especificarse qué datos se registran en auditoría (sin PAN completo).
- Falta referencia a requerimientos de SCA (Strong Customer Authentication) bajo PSD2 si aplica la jurisdicción europea.

ITERACIÓN 2 — REVISIÓN

Prompt al agente:

```
[contexto original] + [respuesta inicial] + [crítica]
```

Genera una versión revisada de la política incorporando todas las correcciones identificadas en la crítica.

Respuesta revisada:

POLÍTICA DE ACCESO — API DE PAGOS v2

1. Autenticación

- Todos los clientes usan OAuth 2.0 con tokens JWT firmados con RS256 (clave mínima 2048 bits).
- Tokens de acceso: vigencia de 30 minutos. Refresh tokens: vigencia de 8 horas, rotación automática en cada uso.
- HTTPS obligatorio (TLS 1.2 mínimo). Para clientes institucionales: mutual TLS con certificado de cliente válido.
- Bloqueo temporal de cliente tras 5 intentos fallidos en 10 minutos. Desbloqueo manual por administrador o automático a las 2 horas.
- Proceso de revocación de emergencia: invalidación inmediata de todos los tokens del cliente mediante endpoint de revocación.

2. Autorización (RBAC)

- Rol "lectura": consultar estado de transacciones propias.
- Rol "pago": iniciar transacciones de pago (requiere autenticación 2FA adicional para montos superiores a umbral configurable).
- Rol "administración": gestionar clientes, configurar límites, acceder a reportes agregados.
- Las operaciones de pago requieren verificación adicional por SCA cuando aplique PSD2.

3. Rate Limiting (por cliente y por tipo de operación)

- Consultas: 5000 requests/hora.
- Iniciación de pagos: 100 requests/hora (límite más restrictivo por impacto financiero).
- Administración: 500 requests/hora.
- Alertas automáticas al 80% del límite; bloqueo temporal al 100%.

4. Auditoría

- Se registra todo acceso con: timestamp, cliente ID, endpoint, resultado, IP de origen. Sin datos de PAN completo (solo últimos 4 dígitos).
- Retención de logs: mínimo 12 meses en línea, 7 años en archivo (PCI DSS).
- Logs inmutables: no modificables ni eliminables por ningún rol incluyendo

```
administración.  
- Alertas en tiempo real para: autenticaciones fallidas repetidas, accesos  
fuera de horario habitual, volúmenes de transacción inusualmente altos.
```

Qué puede y qué no puede detectar la reflexión

La reflexión es efectiva para detectar:

- Omisiones de elementos que deberían estar presentes (la crítica puede revisar una lista de requisitos conocidos)
- Inconsistencias internas en el output (contradicciones entre partes del mismo documento)
- Violaciones de restricciones explícitas (longitud, formato, inclusión de elementos requeridos)
- Brechas de cobertura (temas mencionados en el prompt que no aparecen en el output)

La reflexión no puede detectar de forma confiable:

- Errores factuales en dominios donde el evaluador tiene el mismo conocimiento incorrecto que el generador
- Errores de razonamiento matemático o lógico donde el modelo produce resultados incorrectos con confianza alta
- Violaciones de requisitos no especificados en el prompt (el modelo no puede saber lo que no se le dijo)

Esta limitación es la razón por la que la verificación externa — examinada en la siguiente sección — es un complemento necesario, no un sustituto, de la reflexión interna.

El evaluador separado

Una mejora significativa al patrón de reflexión básico es usar un evaluador separado: un modelo diferente, con un prompt diferente, que actúa como crítico independiente. Esto reduce el riesgo de que el evaluador comparta los puntos ciegos del generador.

En práctica, incluso usar el mismo modelo con un prompt de evaluación muy diferente al prompt de generación produce evaluaciones más útiles que pedir al modelo que evalúe su propio output directamente. El prompt de evaluación debe:

- Especificar criterios de evaluación explícitos

- Pedir al modelo que busque activamente deficiencias, no que confirme calidad
- Proporcionar ejemplos de los tipos de problemas que debe encontrar
- Instruir al modelo para que sea específico, no genérico ("la sección X omite Y" en lugar de "podría ser más completo")

Nota del arquitecto

La reflexión no es gratuita. Agrega latencia y costo en proporción al número de iteraciones. El AI Engineer debe definir a priori cuántas iteraciones son válidas y qué criterio determina cuándo la reflexión ha convergido. Un output aceptable después de dos iteraciones es mejor que un output ligeramente mejor después de cinco iteraciones si la latencia adicional tiene un costo en la experiencia del usuario o en el costo operativo del sistema.

La siguiente sección examina la verificación de resultados: el mecanismo externo que complementa la reflexión interna cuando la tarea requiere garantías de corrección que el modelo solo no puede proporcionar.

Capítulo 10 — Planificación y Razonamiento

Sección 07: Verificación y corrección de resultados

La reflexión, como se examinó en la sección anterior, es un mecanismo de autoevaluación que tiene límites estructurales: el mismo sistema que generó el resultado es el que lo evalúa. Para resultados donde la corrección importa — código que debe ejecutarse, respuestas que serán citadas como fuente de verdad, planes que guiarán decisiones de negocio — la verificación debe involucrar mecanismos externos al modelo.

La verificación externa no reemplaza la reflexión; la complementa. La reflexión mejora la coherencia y la completitud. La verificación externa garantiza la corrección en dimensiones que el modelo no puede verificar por sí mismo.

Taxonomía de estrategias de verificación

La estrategia de verificación correcta depende del tipo de output. Un enfoque único no funciona para todos los casos. A continuación se presentan las estrategias principales organizadas por tipo de output.

Tipo 1: Código y outputs computacionales

El problema: Un modelo puede generar código que parece sintácticamente correcto, pero que falla en ejecución, produce resultados incorrectos o no maneja casos borde.

Estrategia de verificación:

NIVEL 1 — VERIFICACIÓN SINTÁCTICA

Pasar el código por un linter o parser del lenguaje objetivo.

Costo: negligible. Detecta: errores de sintaxis.

No detecta: errores lógicos, casos borde.

NIVEL 2 — VERIFICACIÓN DE EJECUCIÓN

Ejecutar el código en un sandbox aislado.

Verificar que termina sin excepción con un input de prueba básico.

Costo: bajo. Detecta: errores de runtime obvios.

No detecta: resultados incorrectos, casos borde específicos.

NIVEL 3 — VERIFICACIÓN CON TESTS

Ejecutar el código contra una suite de tests unitarios predefinida.

Costo: medio. Detecta: errores lógicos para los casos testeados.

No detecta: casos fuera de la suite de tests.

NIVEL 4 — VERIFICACIÓN POR ANÁLISIS

Usar otro LLM con un prompt de revisión de código para identificar problemas que los tests no cubren.

Costo: alto. Detecta: problemas de diseño, patrones de error comunes.

No detecta: errores factuales, problemas de dominio muy específicos.

Ejemplo de flujo:

Un agente genera una función Python para calcular el interés compuesto de un préstamo. El sistema de verificación:

1. Pasa el código por `pyflakes` (nivel 1): sin errores de sintaxis.
2. Ejecuta la función con un input conocido — 10.000 de capital, 5% de tasa, 3 años — y compara el resultado contra el valor calculado manualmente (nivel 2): el resultado es correcto.
3. Ejecuta los tests unitarios que incluyen casos con tasa 0%, periodos = 1, capital negativo (nivel 3): la función falla con capital negativo — no valida el input.
4. El sistema retroalimenta el error al agente para que corrija la función.

Tipo 2: Respuestas factuales con fuente de verdad disponible

El problema: Un modelo puede afirmar datos que son incorrectos, desactualizados o fabricados (alucinación), incluso con aparente confianza.

Estrategia de verificación:

La verificación factual requiere una fuente de verdad externa. Si esa fuente existe, el proceso es:

1. El agente genera la respuesta.
2. El sistema extrae las afirmaciones verificables de la respuesta
(puede ser una segunda llamada al modelo: "extrae todas las afirmaciones de hechos específicos de este texto").
3. Para cada afirmación, el sistema busca evidencia en la fuente de verdad
(base de datos, documentos RAG, API de datos).
4. Las afirmaciones con evidencia se marcan como verificadas.
5. Las afirmaciones sin evidencia se marcan como no verificadas.
6. La respuesta final incluye solo afirmaciones verificadas, o incluye las no verificadas con una indicación explícita de que no fueron verificadas.

Cuándo no se puede verificar factualmente: Si no existe una fuente de verdad accesible, la verificación factual no es posible mediante este mecanismo. En ese caso, la estrategia es limitar el alcance de las afirmaciones del modelo: instruir al sistema para que solo afirme lo que puede soportar con citas de los documentos del contexto, y que marque explícitamente como "no verificado" cualquier conocimiento que proviene de los pesos del modelo.

Tipo 3: Planes de acción y workflows

El problema: Un plan generado por el modelo puede ser estructuralmente correcto pero incluir pasos que no son ejecutables — porque la herramienta requerida no está disponible, porque los permisos necesarios no existen, o porque el output de un paso no es compatible con el input que espera el siguiente paso.

Estrategia de verificación:

VERIFICACIÓN DE VIABILIDAD

Para cada paso del plan:

- ¿La herramienta requerida está disponible en el sistema?
- ¿El agente tiene los permisos necesarios para usar esa herramienta?
- ¿El formato del output del paso anterior es compatible con el formato de input que espera este paso?

VERIFICACIÓN DE COMPLETITUD

- ¿El plan cubre todos los requisitos de la tarea?
- ¿Hay pasos de verificación después de acciones críticas?
- ¿Hay un mecanismo de rollback si un paso falla?

VERIFICACIÓN DE COHERENCIA

- ¿El plan termina en un estado que satisface la tarea?
- ¿Hay dependencias cíclicas entre pasos?
- ¿El orden de los pasos es correcto (no se usa output antes de generarlo)?

Esta verificación puede hacerse de forma parcialmente automatizada (verificación de herramientas disponibles mediante lookup en un registro de herramientas) y de forma asistida por LLM (verificación de coherencia y completitud mediante una llamada de revisión).

Tipo 4: Outputs estructurados (JSON, XML, tablas)

El problema: El modelo puede generar un JSON que no cumple el esquema esperado — campos ausentes, tipos incorrectos, valores fuera de rango — lo que rompe el sistema downstream que consume ese output.

Estrategia de verificación:

Esta es la verificación más mecánica y la más fácil de implementar. Antes de pasar el output a cualquier sistema downstream:

1. Validar el JSON/XML contra el esquema esperado (JSON Schema, XSD).
2. Verificar que los campos obligatorios están presentes.
3. Verificar que los valores están dentro de rangos válidos para el dominio.

4. Si la validación falla, retroalimentar el error al modelo con el esquema esperado y el mensaje de error, y pedir una corrección.

La mayoría de los frameworks de agentes modernos incluyen soporte para generación de outputs estructurados con validación automática (structured outputs, tool calling con schemas). Usar esta capacidad en lugar de parsear texto libre elimina la mayoría de los errores de tipo 4.

LLM-as-judge: verificación asistida por modelo

Cuando la verificación automática no es posible — el output es texto no estructurado y no existe una fuente de verdad mecánicamente accesible — una opción es usar un segundo modelo como evaluador. Este patrón se conoce como LLM-as-judge.

Principios de diseño del LLM-as-judge:

- El modelo evaluador recibe: el output original, los criterios de evaluación explícitos y, si es posible, ejemplos de outputs buenos y outputs malos.
- El prompt del evaluador debe pedir una evaluación estructurada: puntaje en cada criterio, justificación específica para cada puntaje.
- El evaluador debe ser instruido para buscar problemas activamente, no para confirmar calidad.
- Si es posible, usar un modelo diferente al generador para el evaluador. Si el mismo modelo se usa, el prompt debe ser significativamente diferente.

Limitación fundamental: El LLM-as-judge hereda los puntos ciegos del modelo evaluador. Para cualquier error que el modelo evaluador no pueda detectar, este mecanismo no ayuda. Es una capa adicional de calidad, no una garantía de corrección.

Combinando verificación y reflexión

El flujo de alta calidad en producción combina ambos mecanismos:

```
[Generación] → [Reflexión interna: coherencia y completitud]
               → [Verificación externa: corrección según tipo de output]
               → [Si verificación falla: retroalimentación al agente]
               → [Nueva generación con contexto de error]
               → [Re-verificación]
               → [Output final si verificación pasa]
               → [Escalada a humano si no converge en N iteraciones]
```

Nota del arquitecto

La tentación en producción es confiar en la reflexión interna y omitir la verificación externa para reducir latencia y costo. Este es el error que lleva a sistemas que funcionan bien en demos pero fallan en producción. Para cualquier output que tomará una decisión real — código que se desplegará, respuesta que se presentará como hecho, plan que se ejecutará sobre recursos reales — la verificación externa no es opcional. El costo de la verificación es una fracción del costo de los errores que previene.

La siguiente sección examina cómo estos mecanismos se integran en arquitecturas empresariales de mayor escala.

Capítulo 10 — Planificación y Razonamiento

Sección 08: Arquitecturas de planificación empresarial

Los patrones y técnicas de las secciones anteriores son piezas de construcción. Esta sección examina cómo esas piezas se ensamblan en arquitecturas completas que operan en entornos empresariales reales — con sus restricciones de latencia, costo, regulación y disponibilidad.

Un sistema de planificación empresarial no es solo un agente que razona bien. Es un sistema que razona bien dentro de los límites operacionales del negocio, produce outputs auditables, falla de forma predecible y escala bajo carga.

Las cuatro dimensiones del diseño empresarial

Dimensión 1: Calidad del razonamiento

El sistema debe producir outputs de calidad suficiente para el caso de uso. "Suficiente" es la palabra clave: el nivel de sofisticación del razonamiento debe estar calibrado al nivel de precisión que el caso de uso requiere, no al nivel máximo que la técnica permite.

Un sistema de análisis de documentos legales requiere un nivel diferente de razonamiento que un sistema de clasificación de emails de marketing. La arquitectura de planificación debe reflejar esa diferencia: más iteraciones de reflexión, más capas de verificación, más llamadas al modelo donde la calidad es crítica; menos donde no lo es.

Dimensión 2: Latencia

En entornos empresariales, la latencia no es solo una métrica de experiencia de usuario. Es a menudo un requisito contractual (SLA) o una restricción funcional (el resultado del agente bloquea un proceso upstream).

La relación entre sofisticación del razonamiento y latencia es directa: más llamadas al modelo implica más latencia. El arquitecto debe conocer el presupuesto de latencia del caso de uso y diseñar la arquitectura de razonamiento dentro de ese presupuesto.

Estrategias para gestionar la latencia:

- Paralelizar llamadas al modelo cuando no tienen dependencias entre sí (por ejemplo, evaluar múltiples documentos simultáneamente).
- Usar modelos más pequeños y rápidos para los pasos de evaluación y reflexión donde la precisión completa no es necesaria.
- Cachear outputs de pasos que no cambian entre ejecuciones similares.
- Separar las fases de planificación (puede ser asíncrona) de las fases de acción (puede requerir latencia baja).

Dimensión 3: Auditabilidad

En sectores regulados — finanzas, salud, legal — el sistema debe poder responder a la pregunta: "¿Por qué el agente tomó esta decisión?". La respuesta no puede ser "el modelo lo generó". Debe ser un registro estructurado de los pasos de razonamiento, las herramientas usadas, los datos consultados y los criterios aplicados.

Los sistemas de planificación empresarial deben registrar:

- El input completo de cada llamada al modelo
- El output completo de cada llamada al modelo
- Las herramientas usadas y sus inputs/outputs
- El resultado de cada verificación
- La decisión tomada en cada punto de bifurcación del plan

Esta traza de ejecución no es solo para auditoría: es también la base para depurar el sistema cuando falla y para mejorar el diseño basándose en datos de producción.

Dimensión 4: Escalada y control humano

Ningún sistema de planificación autónoma es apropiado para todos los escenarios. El arquitecto debe definir explícitamente:

- Qué decisiones el agente puede tomar de forma autónoma
- Qué decisiones requieren confirmación humana antes de ejecutarse
- Qué situaciones desencadenan una escalada automática al operador humano
- Cómo se interrumpe el agente si es necesario

El patrón de escalada más robusto es el "human-in-the-loop" configurable: el sistema puede operar en modo completamente autónomo, en modo de confirmación (el humano aprueba cada acción antes de ejecutarse) o en modo de revisión (el agente actúa pero el humano puede

revertir dentro de una ventana de tiempo). El nivel de supervisión se configura según el tipo de acción y el nivel de riesgo.

Arquitectura de referencia

La siguiente arquitectura de referencia combina los cuatro patrones de planificación con las cuatro dimensiones empresariales. Es una estructura de tres capas:

CAPA 1 — PLANIFICACIÓN ESTRATÉGICA

Input: tarea del usuario + contexto de negocio

Proceso: llamada de planificación (CoT) → plan de alto nivel

Output: plan estructurado con pasos, herramientas, criterios de éxito

Latencia: puede ser asíncrona (el usuario espera el plan, no la ejecución)

CAPA 2 — EJECUCIÓN TÁCTICA (por cada paso del plan)

Input: paso del plan + estado actual + memoria de trabajo

Proceso:

- Ejecutar acción (herramienta o llamada al modelo)
- Verificar output según tipo (código / factual / estructurado)
- Si verificación falla → reflexión + corrección (máx. N iteraciones)
- Si no converge → escalar a humano

Output: resultado del paso + actualización de memoria de trabajo

Latencia: sincrónica, debe cumplir SLA

CAPA 3 — SÍNTESIS Y ENTREGA

Input: resultados de todos los pasos + tarea original

Proceso: llamada de síntesis → respuesta final

Output: respuesta al usuario / acción en sistema externo

Auditabilidad: registro completo de capas 1 y 2 disponible

Decisiones de diseño comunes

¿Un modelo o múltiples modelos?

En sistemas de producción, usar un modelo diferente para planificación (donde se necesita razonamiento profundo) y para ejecución de pasos de bajo riesgo (donde se necesita velocidad)

y bajo costo) es una decisión válida y frecuente. El modelo de planificación puede ser mayor y más preciso; el modelo de ejecución de pasos simples puede ser menor y más rápido.

¿Planificación en tiempo real o planificación pre-computada?

Para casos de uso con alta variabilidad de tareas (cada solicitud es diferente), la planificación debe ser en tiempo real. Para casos de uso donde las tareas son variantes de un conjunto limitado de flujos conocidos, los planes pueden pre-computarse y almacenarse, usando la llamada al modelo solo para adaptar el plan almacenado al contexto específico. Esto reduce significativamente la latencia.

¿Cómo gestionar el contexto en sistemas de larga duración?

Los sistemas de planificación que operan durante horas o días (por ejemplo, un agente que gestiona un proceso de semanas) deben gestionar el contexto de forma activa. La ventana de contexto del modelo tiene límites. El sistema debe mantener una memoria de trabajo que incluya solo la información relevante para el estado actual, y archivar el historial completo en almacenamiento externo (cubierto en el capítulo 09).

Nota del arquitecto

El error más común en la transición de prototipo a producción es no planificar para el fallo. En el prototipo, el happy path funciona bien. En producción, el agente inevitablemente encuentra situaciones que no estaban en el conjunto de pruebas: herramientas que no responden, datos con formatos inesperados, tareas ambiguas que el plan no puede completar. El diseño debe incluir, desde el inicio, qué hace el sistema cuando falla — no como caso excepcional, sino como parte del diseño principal.

La siguiente sección examina los patrones que funcionan y los que fallan en producción, con el análisis de por qué.

Capítulo 10 — Planificación y Razonamiento

Sección 09: Patrones y anti-patrones

El conocimiento de los patrones correctos tiene un valor limitado si no se acompaña del conocimiento de los patrones incorrectos y de por qué fallan. Esta sección documenta los anti-patrones más frecuentes en sistemas de planificación y razonamiento en producción, con el análisis de las condiciones que los generan y las decisiones de diseño que los evitan.

Patrones que funcionan

Patrón P1: Razonamiento visible y auditable

El agente produce los pasos de razonamiento como texto explícito antes de producir el output final. Cada paso es un fragmento de texto independiente que puede revisarse y auditarse.

Por qué funciona: El razonamiento explícito reduce la tasa de errores porque el modelo "piensa en voz alta" antes de comprometerse con una respuesta. Los errores en el razonamiento son detectables antes de que lleguen al output final. El sistema de verificación puede revisar la cadena de razonamiento, no solo el output.

Señal de éxito: El equipo puede depurar un output incorrecto mirando el razonamiento y identificando exactamente en qué paso se introdujo el error.

Patrón P2: Verificación por tipo de output

El sistema aplica una estrategia de verificación específica al tipo de output que produce cada paso (código, texto, datos estructurados, plan).

Por qué funciona: Cada tipo de output tiene formas de fallo características y mecanismos de verificación apropiados. Un sistema de verificación genérico no puede detectar los errores específicos de cada tipo. La especialización por tipo incrementa la cobertura de detección de errores.

Patrón P3: Escalada explícita y predefinida

El sistema tiene definidas, en el diseño, las condiciones exactas bajo las cuales escala al operador humano: tipos de error, umbrales de confianza, categorías de decisión de alto riesgo.

Por qué funciona: Sin escalada explícita, los sistemas de planificación intentan resolver todo de forma autónoma, incluyendo situaciones para las que no están diseñados. La escalada predefinida convierte el fallo en un proceso gestionado en lugar de en una crisis.

Patrón P4: Límites de recursos explícitos

El sistema tiene configurados límites explícitos de iteraciones, llamadas al modelo, tiempo de ejecución y costo por tarea.

Por qué funciona: Previene que los sistemas entren en loops indefinidos, generen gastos inesperados o bloqueen recursos por tiempo ilimitado. Los límites son la red de seguridad que convierte un sistema potencialmente inestable en uno predeciblemente acotado.

Anti-patrones que fallan

Anti-patrón AP1: El optimista de una sola llamada

Descripción: El sistema confía en que una sola llamada al modelo producirá un output de calidad suficiente para cualquier tarea, sin pasos de verificación ni reflexión.

Por qué falla: Para tareas simples, esto es correcto y eficiente. Para tareas complejas, el modelo omite pasos de razonamiento, produce respuestas incompletas y comete errores que una segunda llamada de verificación habría detectado. El sistema funciona bien en las demos (las demos suelen usar tareas simples y bien formadas) y falla en producción (donde las tareas son más complejas e impredecibles).

Síntoma característico: Alta tasa de fallos en producción que no aparecía en las pruebas. Los ingenieros no pueden reproducir el fallo porque en el entorno de prueba se usaron inputs más simples.

Corrección: Categorizar las tareas por complejidad y aplicar la arquitectura de razonamiento apropiada para cada categoría. No todas las tareas necesitan reflexión; pero las tareas complejas la necesitan.

Anti-patrón AP2: La reflexión infinita

Descripción: El sistema realiza múltiples iteraciones de reflexión sin un criterio de terminación claro, con la esperanza de que más iteraciones producirán un output mejor.

Por qué falla: La reflexión sin criterio de terminación produce tres problemas: (a) el sistema puede no converger, generando iteraciones indefinidamente sin mejorar el output; (b) más allá de 2-3 iteraciones, la reflexión frecuentemente empeora el output — el modelo se vuelve excesivamente cauteloso, añade calificaciones innecesarias o rompe la coherencia del texto; (c) el costo y la latencia crecen sin límite.

Síntoma característico: El sistema tarda mucho más de lo esperado en producir respuestas. Los outputs tienen calidad irregular: excelentes para algunas entradas, degradados para otras. El costo de la API es mucho mayor al estimado.

Corrección: Definir el criterio de convergencia antes de implementar la reflexión: qué característica del output indica que la reflexión ha sido suficiente. Si no se puede definir un criterio claro, limitar las iteraciones a un número fijo (generalmente 2) y aceptar ese output.

Anti-patrón AP3: El plan que no puede fallar

Descripción: El sistema genera un plan y lo ejecuta paso a paso sin ningún mecanismo de detección de fallo intermedio. Si un paso falla, el sistema intenta continuar con el siguiente paso usando un resultado incompleto o incorrecto.

Por qué falla: Los errores en pasos tempranos se propagan a través de todos los pasos siguientes. El output final puede ser totalmente incorrecto sin que ningún paso individualmente parezca haber fallado. El diagnóstico es extremadamente difícil porque el fallo visible (el output final) está desconectado de la causa (el paso que falló primero).

Síntoma característico: Outputs que son coherentes internamente pero incorrectos globalmente. El sistema no reporta ningún error pero el resultado es inutilizable.

Corrección: Añadir verificación del output de cada paso antes de pasarlo al siguiente. Definir el comportamiento cuando un paso falla: reintento, plan alternativo o escalada. Nunca pasar un resultado de fallo a un paso siguiente como si fuera un resultado exitoso.

Anti-patrón AP4: El agente sin herramientas que ejecuta herramientas

Descripción: El sistema planifica acciones que requieren herramientas no disponibles, o usa herramientas fuera del rango de inputs para los que fueron diseñadas, sin verificar la disponibilidad ni los límites antes de ejecutar.

Por qué falla: El modelo no sabe qué herramientas están disponibles a menos que se le informe explícitamente. Si el prompt de planificación no incluye el catálogo exacto de herramientas disponibles con su descripción de uso y límites, el modelo planificará usando herramientas que no existen o usando herramientas existentes de formas no soportadas. La ejecución fallará en tiempo de ejecución de formas que son difíciles de anticipar.

Síntoma característico: Altas tasas de error en la fase de ejecución. Los logs muestran llamadas a herramientas con parámetros inválidos, herramientas no encontradas, o timeouts inesperados.

Corrección: El prompt de planificación debe incluir siempre el catálogo completo de herramientas disponibles con sus inputs esperados, outputs producidos y limitaciones conocidas. La verificación de viabilidad del plan (sección 07) debe incluir verificar que cada herramienta del plan existe en el catálogo.

Anti-patrón AP5: La verificación circular

Descripción: El sistema usa el mismo modelo con el mismo prompt para generar el output y para verificarlo. El evaluador tiene exactamente los mismos puntos ciegos que el generador.

Por qué falla: Si el modelo cometió un error porque no conoce algo, o porque tiene un sesgo particular en un dominio, el mismo modelo con los mismos sesgos no detectará ese error al evaluarlo. La verificación circular da una falsa sensación de seguridad: el sistema reporta que el output fue verificado, pero la verificación no agregó ninguna garantía real.

Síntoma característico: El sistema reporta alta confianza en sus outputs, pero la tasa de errores en producción es más alta de lo esperado. Los errores que el sistema "verifica" y considera correctos son sistemáticamente del mismo tipo.

Corrección: Para verificación de calidad del output, usar un prompt de evaluación significativamente diferente al prompt de generación. Para verificación factual, usar una fuente de verdad externa. Para verificación de código, usar ejecución real. Para verificación de planes, usar validación mecánica del catálogo de herramientas.

Tabla resumen

Anti-patrón	Síntoma principal	Corrección clave
Optimista de una sola llamada	Fallos en producción no reproducibles en pruebas	Categorizar por complejidad
Reflexión infinita	Latencia alta, costo inesperado, output degradado	Criterio de convergencia + límite de iteraciones
Plan que no puede fallar	Output final incorrecto sin error visible	Verificación por paso
Agente sin herramientas	Errores de ejecución frecuentes	Catálogo de herramientas en el prompt de planificación
Verificación circular	Alta confianza reportada, alta tasa de error real	Evaluador independiente o verificación externa

La siguiente sección aplica todos estos principios a un caso de estudio empresarial completo.

Capítulo 10 — Planificación y Razonamiento

Sección 10: Caso de estudio empresarial

Sistema de análisis y aprobación de préstamos asistido por IA

Una institución financiera de tamaño mediano procesa entre 200 y 400 solicitudes de préstamos personales por día. El proceso actual es manual: un analista de crédito revisa cada solicitud, consulta el bureau de crédito, verifica documentación y emite una recomendación. El tiempo promedio por solicitud es de 45 minutos. La institución busca implementar un sistema de IA que reduzca ese tiempo a menos de 5 minutos, mantenga la calidad de las decisiones y cumpla con la regulación de transparencia de decisiones crediticias vigente en su jurisdicción.

Este caso de estudio examina las decisiones de arquitectura de planificación que hacen posible ese sistema.

Análisis del problema

Complejidad de la tarea: Alta. Una solicitud de préstamo requiere integrar información de múltiples fuentes heterogéneas (datos del solicitante, historial crediticio, documentos de ingreso, contexto macroeconómico), aplicar criterios regulatorios específicos y producir una recomendación justificada.

Calidad requerida: Crítica. Un error puede implicar pérdida financiera (aprobación incorrecta de un préstamo que no se pagará) o daño reputacional y regulatorio (rechazo incorrecto de un solicitante elegible, con potencial de discriminación).

Latencia requerida: Moderada. El analista humano puede revisar el output del sistema antes de la decisión final; no es necesaria una respuesta en tiempo real.

Auditabilidad requerida: Obligatoria. La regulación exige que la institución pueda explicar cualquier decisión crediticia al solicitante que la solicite.

Control humano: El sistema produce recomendaciones, no decisiones. El analista humano tiene la decisión final. El sistema puede operar en modo completamente autónomo solo para solicitudes que caen claramente dentro de criterios predefinidos (aprobación automática o rechazo automático) y con límites de monto.

Decisiones de arquitectura

Patrón de planificación: Plan-and-Execute. La secuencia de pasos es suficientemente estable como para pre-definirse, pero debe adaptarse al estado de cada solicitud.

Número de modelos: Dos. Un modelo mayor y más preciso para el análisis de riesgo y la redacción de la justificación; un modelo menor y más rápido para la extracción de datos estructurados de los documentos.

Verificación: Múltiples capas, específicas por tipo de output.

El plan de análisis de solicitud

El sistema genera el siguiente plan estándar al inicio de cada solicitud. Los pasos varían según el tipo de solicitud y el perfil del solicitante.

PLAN DE ANÁLISIS — SOLICITUD DE PRÉSTAMO

Paso 1: Extracción de datos del solicitante

Acción: Extraer del formulario de solicitud los campos estandarizados.

Herramienta: extractor de formularios (modelo menor).

Output esperado: JSON con campos validados contra esquema.

Criterio de éxito: JSON válido, todos los campos obligatorios presentes.

Paso 2: Consulta al bureau de crédito

Acción: Consultar el historial crediticio del solicitante.

Herramienta: API bureau de crédito (datos externos).

Output esperado: perfil crediticio estructurado.

Criterio de éxito: respuesta recibida sin error; puntaje crediticio presente.

Paso 3: Verificación de documentación de ingresos

Acción: Analizar los documentos de ingreso adjuntos a la solicitud.

Herramienta: analizador de documentos (modelo mayor).

Output esperado: ingreso mensual neto verificado, fuente de ingreso clasificada.

Criterio de éxito: ingreso verificable en al menos dos documentos consistentes.

Paso 4: Cálculo de ratios financieros

Acción: Calcular indicadores clave para la decisión.

Herramienta: calculadora financiera (función determinista, no modelo).

Output esperado: DTI (debt-to-income), LTI (loan-to-income), capacidad de pago.

Criterio de éxito: todos los ratios calculados con los datos de pasos 1-3.

Paso 5: Evaluación de riesgo

Acción: Integrar todos los datos y producir una evaluación de riesgo.

Herramienta: modelo de análisis (modelo mayor, con reflexión).

Output esperado: nivel de riesgo (bajo/medio/alto), factores que lo determinan.

Criterio de éxito: evaluación justificada, coherente con los datos de entrada.

Paso 6: Generación de recomendación

Acción: Producir la recomendación final para el analista humano.

Herramienta: modelo de síntesis (modelo mayor).

```
Output esperado: recomendación (aprobar/rechazar/solicitar información adicional),  
justificación en lenguaje claro, factores decisivos, condiciones (si aplica).  
Criterio de éxito: recomendación coherente con la evaluación del paso 5;  
justificación verificable contra los datos de los pasos 1-4.
```

Ejecución: un caso real

Solicitud: María García solicita un préstamo personal de 25.000 para consolidación de deudas. Ingreso declarado: 3.200 mensuales. Solicitud de plazo: 48 meses.

Paso 1 — Extracción:

El extractor procesa el formulario. JSON generado incluye todos los campos. Verificación de esquema: válido. Tiempo: 8 segundos.

Paso 2 — Bureau de crédito:

Puntaje crediticio: 682/850. Historial: 2 deudas activas (tarjeta de crédito: 4.800 pendiente, préstamo de vehículo: 8.200 pendiente). Sin mora en los últimos 24 meses. Tiempo: 3 segundos (API externa).

Paso 3 — Verificación de ingresos:

El analizador de documentos procesa tres documentos: nómina del último mes (3.180), nómina del mes anterior (3.210), declaración fiscal del año anterior (ingreso anual: 38.400 / 12 = 3.200 mensual). Consistencia: alta. Ingreso verificado: 3.195 (promedio de los tres documentos). Tiempo: 22 segundos.

Paso 4 — Ratios financieros:

Cuota mensual del préstamo solicitado (25.000, 48 meses, tasa 8.5% anual):

→ 616 / mes

Deudas actuales:

Tarjeta de crédito (pago mínimo estimado): 144 / mes

Préstamo de vehículo (cuota conocida): 230 / mes

Total compromisos de deuda tras el nuevo préstamo:

$616 + 144 + 230 = 990$ / mes

DTI (Debt-to-Income): $990 / 3.195 = 31\%$

LTI (Loan-to-Income): $25.000 / (3.195 \times 12) = 0.65$

Capacidad de pago residual: $3.195 - 990 = 2.205$ / mes

Verificación: todos los ratios calculados. DTI del 31% está dentro del umbral aceptable (límite regulatorio: 40%). LTI de 0.65 está dentro del límite institucional (1.0). Tiempo: 2 segundos.

Paso 5 — Evaluación de riesgo (con reflexión):

Generación inicial:

El modelo produce una evaluación preliminar de riesgo bajo-medio, basada en el puntaje crediticio de 682, el historial sin moras y el DTI del 31%.

Reflexión — evaluación de la generación inicial:

El evaluador identifica un factor que la evaluación inicial no consideró: la solicitante tiene dos deudas activas que pretende consolidar con este préstamo. Si el préstamo se aprueba, la institución debe verificar que las deudas actuales serán efectivamente saldadas, de lo contrario el DTI real podría ser más alto que el calculado.

Revisión:

La evaluación revisada mantiene el nivel de riesgo bajo-medio pero añade una condición: la aprobación debe incluir la obligación de presentar comprobantes de liquidación de las deudas existentes dentro de los 30 días siguientes al desembolso.

Paso 6 — Recomendación:

RECOMENDACIÓN PARA EL ANALISTA

Solicitante: María García

Solicitud: Préstamo personal 25.000 / 48 meses

RECOMENDACIÓN: APROBACIÓN CONDICIONAL

FACTORES FAVORABLES:

- Puntaje crediticio 682: sobre el umbral mínimo (650).
- Sin moras en 24 meses: historial de pago consistente.
- DTI proyectado 31%: dentro del límite del 40%.
- Ingresos verificados en tres fuentes consistentes: 3.195/mes.
- Capacidad de pago residual de 2.205/mes: margen adecuado.

FACTORES DE ATENCIÓN:

- La solicitud es para consolidación de deudas. El DTI calculado asume que las deudas actuales serán liquidadas con el préstamo. Si no se liquidan, el DTI real sería del 31% + deudas actuales no liquidadas.

CONDICIÓN DE APROBACIÓN:

- Presentación de comprobantes de liquidación de deudas existentes (tarjeta 4.800 y vehículo 8.200) dentro de los 30 días posteriores al desembolso.
- El desembolso puede realizarse directamente a los acreedores (recomendado) o a la solicitante con verificación posterior.

CRITERIOS REGULATORIOS CUMPLIDOS:

- DTI dentro del límite ($31\% < 40\%$).
- Ingresos verificados documentalmente.
- Puntaje crediticio sobre el mínimo institucional.

Esta recomendación está basada en la información disponible al momento del análisis. La decisión final corresponde al analista responsable.

Resultados del diseño

Tiempo total de análisis: $8 + 3 + 22 + 2 + 35$ (reflexión en paso 5) + 18 (síntesis en paso 6) = 88 segundos.

El objetivo era menos de 5 minutos. Se cumple con margen amplio.

Auditabilidad: Cada paso tiene registrado su input, output y tiempo. La recomendación cita los datos que la fundamentan. El analista puede verificar cualquier dato contra la fuente original.

Calidad: La reflexión en el paso 5 detectó un factor que la evaluación inicial no había considerado — la consolidación de deudas — y lo incorporó como condición. Sin la reflexión, la recomendación habría sido menos precisa.

Control humano: El sistema produce una recomendación, no una decisión. El analista humano tiene toda la información necesaria para aceptar, modificar o rechazar la recomendación.

Lecciones del caso

1. El diseño del plan de ejecución es tan importante como la calidad del modelo. Un plan bien estructurado con pasos verificables produce resultados más confiables que un modelo superior con un plan deficiente.
2. La reflexión debe tener un criterio de activación. En este caso, se activó en el paso de evaluación de riesgo — el paso de mayor impacto y mayor probabilidad de error — y no en los pasos de extracción o cálculo, donde la verificación mecánica es más efectiva.
3. Las herramientas deterministas son superiores al modelo para cálculos numéricos. El paso 4 usa una función de cálculo, no el modelo, para los ratios financieros. Los modelos cometen errores aritméticos; las funciones no.
4. La auditabilidad no es un añadido posterior. Se diseña desde el inicio como parte del output de cada paso.

La siguiente sección presenta el laboratorio práctico donde el lector aplica estos principios a su propio caso de uso.

Capítulo 10 — Planificación y Razonamiento

Sección 11: Laboratorio práctico

Ejercicio: Trazado del árbol de planificación

El objetivo de este laboratorio es que el lector diseñe el árbol de planificación de un agente antes de escribir ningún código. El trazado previo — definir qué piensa el agente primero, qué ejecuta, qué verifica — es la práctica de diseño más importante en sistemas de razonamiento. Un árbol de planificación bien trazado es la especificación del sistema; el código que lo implementa es secundario.

El caso de negocio

Una empresa de comercio electrónico quiere implementar un agente que asista a los analistas de operaciones en la detección de anomalías en sus procesos logísticos. El agente recibe como input un reporte diario de operaciones (datos de pedidos, tiempos de entrega, tasas de error por centro de distribución, datos de devoluciones) y debe producir como output:

1. Un listado de anomalías detectadas, ordenadas por severidad.
2. Para cada anomalía: descripción, datos que la evidencian, posible causa raíz, acción recomendada.
3. Una síntesis ejecutiva de dos párrafos para el responsable de operaciones.

Las herramientas disponibles para el agente son:

- `leer_reporte` : accede al reporte diario de operaciones en formato JSON.
 - `consultar_historico` : consulta datos históricos de hasta 90 días para comparación.
 - `buscar_incidentes` : busca en la base de conocimiento de incidentes previos similares.
 - `calcular_estadisticas` : calcula estadísticas descriptivas sobre series de datos.
 - `generar_alerta` : registra una alerta en el sistema de monitoreo.
-

Parte 1: Diseño del árbol de planificación

El lector debe completar el siguiente árbol de planificación antes de continuar a la parte 2.

Instrucción: Para cada nodo del árbol, definir:

- La acción o decisión que representa el nodo
- La herramienta utilizada (si aplica)
- El output esperado
- El criterio de éxito para ese nodo
- Qué ocurre si el nodo falla

NODO RAÍZ

Input: [definir qué información recibe el agente al inicio]

Tarea: [formular la tarea completa en una frase]

NIVEL 1 — COMPRENSIÓN DEL ESTADO ACTUAL

Nodo 1.1: _____

Herramienta: _____

Output esperado: _____

Criterio de éxito: _____

Si falla: _____

Nodo 1.2: _____

Herramienta: _____

Output esperado: _____

Criterio de éxito: _____

Si falla: _____

NIVEL 2 — DETECCIÓN DE ANOMALÍAS

Nodo 2.1: _____

[completar]

Nodo 2.2: _____

[completar]

Nodo 2.3: _____

[completar]

NIVEL 3 — ANÁLISIS DE CAUSAS

Nodo 3.1: _____

[completar]

NIVEL 4 — SÍNTESIS Y OUTPUT

Nodo 4.1: _____

[completar]

Nodo 4.2: _____

[completar]

Parte 2: Árbol de planificación de referencia

Una vez que el lector ha completado su propio árbol, puede compararlo con el árbol de referencia a continuación. No existe una única respuesta correcta; el árbol de referencia es una implementación válida entre varias posibles.

NODO RAÍZ

Input: reporte diario en JSON + fecha del reporte

Tarea: "Detectar anomalías en el reporte de operaciones del [fecha],
identificar causas probables y producir recomendaciones priorizadas."

NIVEL 1 — COMPRENSIÓN DEL ESTADO ACTUAL

Nodo 1.1: Lectura y estructura del reporte

Herramienta: leer_reporte

Output esperado: JSON con datos del día estructurados

Criterio de éxito: JSON válido, campos de pedidos, tiempos, errores y
devoluciones presentes

Si falla: abortar y notificar — sin datos no hay análisis posible

Nodo 1.2: Obtención del histórico de comparación

Herramienta: consultar_historico (últimos 30 días)

Output esperado: serie temporal de los mismos indicadores

Criterio de éxito: datos disponibles para al menos 20 de los últimos 30 días

Si falla: continuar sin comparación histórica, marcar todos los hallazgos
como "sin contexto histórico"

NIVEL 2 — DETECCIÓN DE ANOMALÍAS

Nodo 2.1: Cálculo de métricas del día actual

Herramienta: calcular_estadisticas

Input: datos del nodo 1.1

Output esperado: media, desviación y percentil de cada indicador para el día

Criterio de éxito: métricas calculadas para todos los indicadores clave

Nodo 2.2: Comparación con histórico

Herramienta: calcular_estadisticas

Input: datos del nodo 1.2

Output esperado: media histórica y umbral de anomalía ($\text{media} \pm 2\sigma$) para
cada indicador

Criterio de éxito: umbrales calculados para todos los indicadores

Nodo 2.3: Identificación de anomalías (modelo – análisis)

Input: métricas del día (2.1) + umbrales históricos (2.2)

Output esperado: lista de indicadores que superan los umbrales, con la magnitud de la desviación y la severidad (alta/media/baja)

Criterio de éxito: lista estructurada, cada anomalía justificada con los datos que la evidencian

Verificación: la lista es coherente con los umbrales calculados en 2.2
(verificación mecánica: cada anomalía declarada supera efectivamente el umbral)

NIVEL 3 – ANÁLISIS DE CAUSAS

Nodo 3.1: Búsqueda de incidentes similares (por cada anomalía)

Herramienta: buscar_incidentes

Input: descripción de cada anomalía

Output esperado: incidentes previos similares con sus causas raíz y resoluciones

Criterio de éxito: búsqueda ejecutada; puede retornar vacío si no hay precedente

Nodo 3.2: Hipótesis de causas raíz (modelo – razonamiento, con reflexión)

Input: cada anomalía + incidentes similares (3.1) + datos del reporte

Output esperado: para cada anomalía, lista de causas raíz posibles ordenadas por probabilidad, con la evidencia que soporta cada hipótesis

Reflexión: el evaluador verifica que las causas son consistentes con los datos disponibles y no contradicen la información del histórico

Criterio de éxito: al menos una hipótesis de causa raíz por anomalía, con evidencia explícita

NIVEL 4 – SÍNTESIS Y OUTPUT

Nodo 4.1: Generación de recomendaciones (modelo – síntesis)

Input: anomalías + causas raíz + incidentes previos (con resoluciones)

Output esperado: para cada anomalía, acción recomendada concreta y responsable sugerido (si se puede inferir del contexto)

Criterio de éxito: cada recomendación es accionable – especifica qué hacer, no solo qué revisar

Nodo 4.2: Síntesis ejecutiva (modelo – redacción)

Input: listado completo de anomalías, causas y recomendaciones

Output esperado: dos párrafos en lenguaje no técnico para el responsable de operaciones: (1) situación general del día, (2) las dos o tres acciones más urgentes

Criterio de éxito: coherente con el listado detallado; sin contradecir ningún hallazgo del análisis

Nodo 4.3: Registro de alertas

Herramienta: generar_alerta

Input: anomalías de severidad alta

Output esperado: confirmación de registro en el sistema de monitoreo

Criterio de éxito: confirmación recibida para cada alerta de severidad alta

Parte 3: Preguntas de análisis

Una vez completado el árbol, responder las siguientes preguntas:

Pregunta 1: En el árbol de referencia, el nodo 2.2 usa `calcular_estadisticas` en lugar de un modelo para calcular los umbrales. ¿Por qué esta decisión es más robusta que usar el modelo para ese cálculo? ¿En qué condiciones podría revertirse esta decisión?

Pregunta 2: El nodo 3.2 incluye reflexión; el nodo 4.1 no. Justifique esta asimetría: ¿qué características de la tarea del nodo 3.2 hacen que la reflexión agregue valor, y qué características del nodo 4.1 hacen que no sea necesaria?

Pregunta 3: Diseñe el mecanismo de escalada para este sistema: ¿qué condiciones deben disparar una escalada al analista humano antes de que el agente complete su ciclo? ¿Qué información debe incluir la escalada?

Pregunta 4: El nodo 1.2 tiene un comportamiento de degradación graceful: si el histórico no está disponible, continúa sin él. Identifique otros dos nodos del árbol donde un comportamiento similar sería apropiado, y describa cómo se degradaría el output en cada caso.

Evaluación del laboratorio

El árbol de planificación diseñado por el lector se evalúa en cinco dimensiones:

1. **Complejidad:** ¿Cubre el árbol todos los pasos necesarios para producir los tres outputs requeridos?
2. **Verificabilidad:** ¿Cada nodo tiene un criterio de éxito explícito y verificable?
3. **Manejo de fallos:** ¿Está definido el comportamiento de cada nodo cuando falla?
4. **Uso apropiado de herramientas:** ¿Las herramientas deterministas se usan para cálculos y las llamadas al modelo se reservan para razonamiento?
5. **Economía de llamadas:** ¿El árbol evita llamadas al modelo donde una herramienta determinista es suficiente?

Un árbol que puntúa bien en estas cinco dimensiones es, en sí mismo, la especificación del sistema. El código que lo implementa es un paso de traducción, no un paso de diseño.

Capítulo 10 — Planificación y Razonamiento

Sección 12: Checklist del AI Engineer

Esta checklist cubre las decisiones de diseño críticas para sistemas de planificación y razonamiento en producción. No es una lista de verificación de implementación — es una lista de decisiones de arquitectura que deben estar resueltas antes de que el sistema entre en producción.

Bloque A: Diseño del razonamiento

A1. ¿Está definido el patrón de planificación para cada tipo de tarea?

- Para cada tipo de solicitud que el sistema procesará, ¿hay una decisión explícita de qué patrón usar (simple / secuencial / iterativo / ramificado)?
- ¿Esa decisión está documentada y es auditable?

A2. ¿El contexto de planificación es completo?

- ¿El prompt de planificación incluye el catálogo completo de herramientas disponibles con sus inputs, outputs y limitaciones?
- ¿El prompt incluye los criterios de éxito de la tarea?
- ¿El prompt incluye las restricciones de dominio relevantes (límites regulatorios, umbrales del negocio, restricciones de acceso a datos)?

A3. ¿El razonamiento es visible?

- ¿El sistema produce los pasos de razonamiento como texto explícito que puede ser auditado?
- ¿Esos pasos se registran junto con el output final?
- ¿Un analista humano puede seguir la cadena de razonamiento del agente hasta el output?

A4. ¿Las herramientas deterministas se usan para cálculos?

- ¿Los cálculos numéricos, la validación de esquemas y las consultas a bases de datos son herramientas deterministas, no llamadas al modelo?
- ¿El modelo se reserva para las tareas que requieren razonamiento sobre lenguaje natural: análisis, síntesis, evaluación?

Bloque B: Reflexión y evaluación

B1. ¿Está definida la política de reflexión?

- ¿Para qué pasos del sistema se activa la reflexión?
- ¿Cuántas iteraciones máximas de reflexión están permitidas?
- ¿Cuál es el criterio de convergencia que determina cuándo la reflexión es suficiente?

B2. ¿El evaluador es suficientemente independiente del generador?

- ¿El prompt del evaluador es significativamente diferente al prompt del generador?
- Si se usa el mismo modelo para generar y evaluar, ¿hay mecanismos adicionales para compensar los puntos ciegos compartidos?
- ¿El evaluador tiene una lista explícita de tipos de problemas que debe buscar, en lugar de una instrucción genérica de "evaluar la calidad"?

B3. ¿La reflexión tiene criterio de activación?

- ¿La reflexión se activa siempre, o solo en los pasos de alto impacto donde el valor de la iteración justifica el costo?
 - ¿Hay una clasificación de pasos por nivel de riesgo que determine cuándo se aplica reflexión y cuándo no?
-

Bloque C: Verificación de outputs

C1. ¿Hay una estrategia de verificación por tipo de output?

- Para cada tipo de output que produce el sistema (código, texto estructurado, plan, respuesta factual), ¿hay una estrategia de verificación específica?
- ¿Esa estrategia está implementada y se activa automáticamente?

C2. ¿La verificación de código incluye ejecución?

- Para outputs de código, ¿el sistema ejecuta el código en un sandbox antes de entregarlo?
- ¿El resultado de la ejecución se usa para retroalimentar al agente si el código falla?

C3. ¿La verificación factual usa fuentes externas?

- Para outputs con afirmaciones factuales, ¿hay una fuente de verdad externa que el sistema consulta?
- ¿Las afirmaciones no verificadas están marcadas como tales en el output?

C4. ¿Los outputs estructurados se validan contra un esquema?

- ¿El sistema valida JSON/XML contra el esquema esperado antes de pasarlo a sistemas downstream?
 - ¿Un fallo de validación desencadena retroalimentación automática al modelo?
-

Bloque D: Control de recursos y fallos**D1. ¿Hay límites explícitos de recursos?**

- ¿Está definido el número máximo de iteraciones del ciclo de planificación?
- ¿Está definido el número máximo de llamadas al modelo por tarea?
- ¿Hay un timeout absoluto por tarea?
- ¿Hay un límite de costo por tarea (en términos de tokens o de costo de API)?

D2. ¿Hay comportamiento de degradación graceful?

- ¿Está definido qué hace el sistema cuando una herramienta no está disponible?
- ¿Está definido qué hace el sistema cuando una fuente de datos externa no responde?
- ¿El sistema puede producir un output de menor calidad (con indicación explícita de la degradación) en lugar de fallar completamente?

D3. ¿Hay detección de estancamiento?

- ¿El sistema detecta cuando el agente ejecuta la misma acción repetidamente sin avanzar?
 - ¿Esa detección desencadena un mecanismo de salida (escalada o aborto)?
-

Bloque E: Auditabilidad y control humano**E1. ¿El sistema registra trazas completas?**

- ¿Se registra el input y output de cada llamada al modelo?
- ¿Se registra el input y output de cada llamada a herramientas?
- ¿Se registra el resultado de cada verificación?
- ¿La traza completa está disponible para auditoría post-hoc?

E2. ¿Está definida la política de escalada?

- ¿Hay una lista explícita de condiciones que desencadenan escalada al operador humano?
- ¿La notificación de escalada incluye la información que el operador necesita para tomar una decisión?
- ¿El sistema puede detenerse y esperar la respuesta del operador sin perder el estado?

E3. ¿Hay separación clara entre recomendación y decisión?

- ¿El sistema produce recomendaciones, no decisiones autónomas, para las operaciones de alto impacto?
 - ¿El analista humano tiene toda la información necesaria para aceptar, modificar o rechazar la recomendación?
 - ¿Los criterios que determinan qué es "alto impacto" están documentados y son revisables?
-

Señales de alerta

Un sistema de planificación que no puede responder afirmativamente a las siguientes preguntas debe ser revisado antes de pasar a producción:

- ¿Puede el equipo explicar, paso a paso, cómo el agente llegó a cualquier output específico?
 - ¿El sistema tiene un mecanismo de parada de emergencia que un operador humano puede activar?
 - ¿El sistema ha sido probado con inputs que producen fallos en las herramientas y en los pasos de verificación?
 - ¿El costo máximo posible por tarea está acotado y dentro de los parámetros económicos del servicio?
 - ¿Hay un proceso definido para revisar y actualizar el sistema cuando los modelos subyacentes cambian?
-

Capítulo 10 — Planificación y Razonamiento

Sección 13: Resumen del capítulo

Lo que el capítulo estableció

El capítulo 10 estudió los mecanismos internos que habilitan la planificación y el razonamiento en sistemas de IA. El punto de partida fue conceptual y fue deliberado: antes de diseñar un sistema de razonamiento, el AI Engineer necesita entender con precisión qué es lo que el LLM hace cuando razona.

El razonamiento en un LLM es predicción estadística de tokens, no inferencia lógica formal. Esta distinción tiene tres consecuencias prácticas que deben guiar cada decisión de diseño: el modelo puede equivocarse con plena confianza, el contexto determina la calidad del razonamiento, y el modelo no tiene acceso introspectivo privilegiado a sus propios procesos. El arquitecto del sistema es quien debe estructurar el contexto, añadir verificación externa y gestionar la corrección donde el modelo solo no es suficiente.

Los cuatro patrones de planificación

El capítulo presentó una taxonomía de cuatro patrones que cubren el espacio de diseño de sistemas de razonamiento:

Patrón simple: Una llamada produce el output completo. Correcto para tareas de baja complejidad donde el modelo tiene alta precisión sin razonamiento multi-paso.

Patrón secuencial: Una cadena de llamadas donde cada output alimenta el input siguiente. Correcto para tareas con estructura de pipeline donde los pasos intermedios son verificables de forma independiente.

Patrón iterativo (con reflexión): El output se evalúa y revisa en ciclos. Correcto para tareas de alta calidad requerida donde los errores son detectables con una segunda revisión. Requiere criterio de convergencia y límite de iteraciones.

Patrón ramificado (Tree of Thoughts): Múltiples ramas de razonamiento se generan, evalúan y podan. Correcto para problemas de optimización donde el primer enfoque intuitivo es frecuentemente subóptimo. Costo computacional alto; se justifica cuando el valor del output es suficientemente alto.

Las técnicas fundamentales

Chain of Thought arquitectónico: Más que una instrucción de prompting, es la decisión de materializar los pasos de razonamiento como outputs separados, incorporados al contexto de las llamadas siguientes. Hace el razonamiento auditable, mejora la precisión en tareas multi-paso y permite intervención en puntos específicos del proceso.

Tree of Thoughts: Una arquitectura de búsqueda sobre un árbol de estados de razonamiento. Requiere generación de múltiples pensamientos, evaluación de cada uno y selección del más prometedor. El costo real en llamadas al modelo crece con el factor de ramificación y la profundidad; en práctica se usa con árboles poco profundos y poda agresiva.

Planificación iterativa y Plan-and-Execute: La planificación dinámica decide el siguiente paso en cada iteración. Plan-and-Execute separa la planificación de la ejecución, haciendo el plan inspeccionable antes de ejecutarse. La replanificación dinámica combina ambos enfoques cuando el entorno es suficientemente incierto.

Reflexión: El ciclo de generación-evaluación-revisión mejora la coherencia y completitud. Puede detectar omisiones, inconsistencias internas y brechas de cobertura. No puede detectar errores factuales donde el evaluador comparte los puntos ciegos del generador.

Las estrategias de verificación

La verificación externa es el complemento necesario de la reflexión interna. La estrategia correcta depende del tipo de output:

- **Código:** verificación sintáctica, ejecución en sandbox, suite de tests.
- **Respuestas factuales:** contraste contra fuente de verdad externa; marcado explícito de afirmaciones no verificadas.
- **Planes de acción:** verificación de disponibilidad de herramientas, coherencia de dependencias, completitud de cobertura.

- **Outputs estructurados:** validación contra esquema JSON/XSD antes de pasar a sistemas downstream.

El LLM-as-judge es una herramienta válida cuando la verificación automática no es posible, con la limitación de que el evaluador hereda los puntos ciegos del modelo generador.

Las cuatro dimensiones empresariales

Más allá de la calidad técnica del razonamiento, los sistemas empresariales deben atender cuatro dimensiones simultáneamente:

1. **Calidad:** calibrada al nivel que el caso de uso requiere, no al máximo que la técnica permite.
 2. **Latencia:** diseñada dentro del presupuesto de tiempo del caso de uso, con paralelización y caching donde sea posible.
 3. **Auditabilidad:** trazas completas de cada llamada, herramienta y verificación, disponibles para auditoría regulatoria y depuración operacional.
 4. **Control humano:** escalada predefinida, separación entre recomendación y decisión, capacidad de intervención en cualquier punto del ciclo.
-

Los cinco anti-patrones críticos

El capítulo identificó los patrones de diseño que más frecuentemente producen fallos en producción:

1. El optimista de una sola llamada, que funciona en demos y falla en producción.
 2. La reflexión infinita, que degrada el output en lugar de mejorarlo más allá de 2-3 iteraciones.
 3. El plan que no puede fallar, donde los errores de pasos tempranos se propagan invisiblemente.
 4. El agente sin herramientas, que planifica acciones que no puede ejecutar.
 5. La verificación circular, que da falsa seguridad sin agregar garantías reales.
-

El principio unificador

El principio que unifica todo el capítulo es este: el razonamiento de un LLM es el resultado del contexto que el arquitecto diseña, no una propiedad intrínseca del modelo. El AI Engineer que entiende esto diseña el contexto con precisión, añade verificación donde el modelo no es confiable, implementa reflexión donde la calidad lo justifica, y construye mecanismos de control donde el sistema opera sobre decisiones de alto impacto.

El capítulo 11 aplica estos principios a un dominio concreto: el desarrollo de software asistido por IA, donde la planificación y el razonamiento se despliegan sobre el problema de entender, generar y mantener código.

Capítulo 10 — Planificación y Razonamiento

Sección 14: Autoevaluación

Las siguientes preguntas evalúan la comprensión de los conceptos fundamentales del capítulo. Algunas tienen una respuesta correcta precisa; otras son preguntas de diseño con múltiples respuestas válidas. Para estas últimas, lo que importa es la calidad del razonamiento que las sustenta.

Preguntas de comprensión conceptual

1. Un colega afirma: "Nuestro agente usa GPT-4, así que razona mejor que los humanos en tareas analíticas." ¿Qué aspectos de esta afirmación son imprecisos? ¿Cómo describirías de forma más precisa lo que el modelo hace cuando produce un análisis?
 2. Explica, en tus propias palabras, por qué la calidad del razonamiento de un LLM depende principalmente del diseño del contexto y no solo de la capacidad del modelo.
 3. ¿Cuál es la diferencia fundamental entre la reflexión interna (el agente evalúa su propio output) y la verificación externa (el sistema verifica el output contra una fuente de verdad)? ¿Qué clases de errores detecta cada una que la otra no puede?
 4. Un sistema usa Tree of Thoughts con 3 niveles y factor de ramificación 3 en todos los niveles, evaluando todas las ramas antes de podar. Estima el número de llamadas al modelo necesarias. ¿En qué condiciones justificarías ese costo en un sistema de producción?
 5. El anti-patrón "verificación circular" implica usar el mismo modelo con el mismo prompt para generar y evaluar un output. ¿Por qué el uso de un prompt de evaluación significativamente diferente (aunque sea el mismo modelo) mejora la situación? ¿Qué limitación fundamental persiste?
-

Preguntas de diseño

6. Eres el AI Engineer responsable de un sistema que genera respuestas a preguntas frecuentes de clientes sobre una política de devoluciones. Las respuestas incorrectas tienen un costo bajo (el cliente puede contactar soporte para aclarar). Diseña la arquitectura de razonamiento más apropiada para este sistema, justificando cada decisión.

7. El mismo caso del punto 6, pero ahora el sistema genera respuestas sobre el estado de reclamaciones de seguros de salud. Una respuesta incorrecta puede resultar en que el paciente no reciba atención médica necesaria. ¿Cómo cambia la arquitectura? ¿Qué mecanismos adicionales añades?

8. Un agente que planifica la compra de materias primas para una empresa manufacturera generó el siguiente plan:

```
Paso 1: Consultar inventario actual de materias primas.  
Paso 2: Determinar necesidades de producción para las próximas 4 semanas.  
Paso 3: Calcular la diferencia entre inventario y necesidades.  
Paso 4: Emitir órdenes de compra para cubrir la diferencia.
```

Identifica los problemas de este plan desde la perspectiva del capítulo. ¿Qué pasos faltan? ¿Qué verificaciones son necesarias? ¿Qué escaladas deben definirse?

9. Un sistema de planificación tiene la siguiente tasa de error en producción: 2% de las tareas producen un output incorrecto que llega al usuario. El equipo propone agregar un paso de reflexión (una iteración de evaluación y revisión) que, según sus pruebas, reduciría la tasa de error al 0.8%. Cada reflexión añade 3 segundos de latencia y 0.05 USD de costo por tarea. El sistema procesa 10.000 tareas por día. El costo de un output incorrecto es de 5 USD en gestión del reclamo. ¿Vale la pena añadir la reflexión? Muestra el análisis económico.

10. Diseña el mecanismo de escalada para el sistema de análisis de solicitudes de préstamo del caso de estudio (sección 10). Define: (a) las condiciones específicas que desencadenan escalada, (b) la información que debe incluir la notificación al analista, (c) qué hace el sistema mientras espera la respuesta del analista, (d) qué ocurre si el analista no responde en el plazo definido.

Preguntas de análisis de casos

11. Un sistema de generación de informes financieros produce informes de 20 páginas a partir de datos de múltiples fuentes. El equipo detecta que el 15% de los informes contienen al menos un dato incorrecto. ¿Cuál de los anti-patrones del capítulo es más probable que esté causando este problema? ¿Cómo lo diagnosticarías y cómo lo corregirías?

12. Un agente de soporte técnico usa planificación iterativa con un límite de 20 iteraciones. En producción, el 8% de las sesiones alcanzan el límite de iteraciones sin resolver el problema del usuario. El equipo quiere aumentar el límite a 40 para darle más oportunidades al agente de resolver. ¿Cuál es el problema con esta solución? ¿Qué analizarías antes de decidir si aumentar el límite es la respuesta correcta?

Respuestas orientativas para preguntas seleccionadas

Respuesta orientativa — Pregunta 4:

Nivel 1: 3 generaciones + 1 evaluación = 4 llamadas.

Nivel 2: $3 \times 3 = 9$ generaciones + 1 evaluación = 10 llamadas (si se expanden todas las ramas del nivel 1).

Nivel 3: $9 \times 3 = 27$ generaciones + 1 evaluación = 28 llamadas (si se expanden todas las ramas del nivel 2).

Total: $4 + 10 + 28 = 42$ llamadas al modelo.

Este costo se justifica cuando: (a) el valor del output es suficientemente alto (decisiones estratégicas de alto impacto), (b) el primer enfoque intuitivo del modelo es frecuentemente subóptimo en este dominio específico, (c) la latencia adicional es aceptable para el caso de uso (análisis que se realiza overnight, no en tiempo real).

Respuesta orientativa — Pregunta 9:

Costo diario actual de errores: $10.000 \times 0.02 \times 5 = 1.000$ USD/día.

Costo diario con reflexión — errores: $10.000 \times 0.008 \times 5 = 400$ USD/día.

Ahorro en errores: 600 USD/día.

Costo adicional de la reflexión: $10.000 \times 0.05 = 500$ USD/día.

Resultado: el ahorro en errores (600 USD) supera el costo de la reflexión (500 USD). El balance neto es positivo en 100 USD/día. La reflexión se justifica económicamente. Consideración adicional: el impacto de 3 segundos adicionales de latencia en la experiencia

del usuario debe evaluarse separadamente — puede ser un factor determinante si el sistema requiere respuesta en tiempo real.

Capítulo 10 — Planificación y Razonamiento

Sección 15: Transición al Capítulo 11

Lo que el capítulo 10 construyó

El capítulo 10 estableció la base de la que depende todo el Context Engineering aplicado: la comprensión de cómo un LLM razona, qué estructura de contexto induce razonamiento de calidad, y cómo el arquitecto del sistema puede verificar, refinar y controlar ese razonamiento de forma sistemática.

Los mecanismos estudiados — Chain of Thought, planificación iterativa, reflexión, verificación por tipo de output, escalada controlada — no son técnicas específicas de un dominio. Son principios de diseño generalizables. Se aplican igual a un sistema que analiza solicitudes de crédito que a uno que gestiona inventarios, coordina operaciones logísticas o diagnostica incidentes técnicos.

El puente hacia el Capítulo 11

El capítulo 11 aplica exactamente estos principios a un dominio específico que tiene características particulares: el desarrollo de software asistido por IA.

El desarrollo de software es, en términos de razonamiento, uno de los dominios más ricos y más exigentes para un sistema de IA. Requiere:

- **Comprensión profunda del contexto:** el código no existe de forma aislada; vive en un ecosistema de módulos, dependencias, convenciones de equipo y decisiones de diseño previas. El agente que asiste al desarrollo necesita entender ese ecosistema antes de actuar sobre él.
- **Planificación multi-paso:** un cambio en el código raramente es atómico. Implica entender el estado actual, identificar qué debe cambiar, evaluar el impacto de ese cambio en el resto del sistema, hacer el cambio, verificar que funciona y verificar que no rompió nada que funcionaba antes. Cada uno de esos pasos es una instancia de los patrones de planificación del capítulo 10.

- **Verificación rigurosa:** el código es uno de los pocos dominios donde la verificación automática es directamente accesible: ejecutar el código y verificar su output es una forma de verificación objetiva que no depende de un evaluador subjetivo. El capítulo 11 construye sobre esta propiedad del dominio.
- **Reflexión sobre el output:** el agente que genera código puede revisarlo antes de entregarlo, evaluar si cubre los casos borde especificados, verificar si sigue las convenciones del proyecto y corregir los problemas identificados. El patrón de reflexión del capítulo 10 tiene en el desarrollo de software una de sus aplicaciones más naturales.

Lo que el lector verá en el Capítulo 11

El capítulo 11 no es un capítulo sobre cómo usar herramientas de IA para escribir código. Es un capítulo sobre cómo diseñar sistemas de Context Engineering que asistan el desarrollo de software de forma confiable, mantenible y alineada con las prácticas de ingeniería del equipo.

El lector verá cómo el contexto de un agente de desarrollo se construye a partir de múltiples fuentes: el código existente del proyecto, las convenciones documentadas del equipo, el resultado de los tests, los mensajes de error del compilador, la especificación del cambio requerido. La calidad del agente depende de la calidad de ese contexto — exactamente el mismo principio que el capítulo 10 estableció para sistemas de razonamiento general.

Verá también cómo el ciclo de planificación-ejecución-verificación del capítulo 10 se instancia concretamente en el ciclo de desarrollo: el agente entiende la tarea, produce un plan de cambio, implementa los cambios, ejecuta los tests, evalúa los resultados y itera hasta que el código es correcto. Cada fase de ese ciclo es una instancia de los mecanismos estudiados en este capítulo.

La continuidad del módulo

Los capítulos 08, 09 y 10 conforman el núcleo técnico del módulo de Context Engineering:

- El capítulo 08 estableció la estructura del agente: cómo percibe, planifica y actúa.
- El capítulo 09 estableció la coordinación entre agentes: cómo múltiples agentes colaboran para resolver problemas que ninguno puede resolver solo.
- El capítulo 10 estableció los mecanismos internos del razonamiento: cómo el agente razona sobre la tarea, verifica sus resultados y corrige sus errores.

El capítulo 11 es la síntesis aplicada: todos esos mecanismos, operando sobre el dominio del desarrollo de software. Es también la transición del módulo de Context Engineering hacia los

módulos de aplicación del libro, donde los principios generales se confrontan con los requisitos específicos de dominios industriales concretos.

El lector que llega al capítulo 11 trae consigo un modelo mental completo de cómo funcionan los sistemas de IA: cómo perciben, cómo coordinan, cómo razonan. El capítulo 11 pone ese modelo a trabajar.